



UNIVERSIDAD
Gastón Dachary

TRABAJO FINAL INTEGRADOR

Sistema de Alquiler de Vehículos

Cátedra: Tecnologías de Bases de Datos

Docentes:

Ing. Suenaga, Roberto

Ing. Barreyro, Enrique

Ciclo Lectivo: Primer Cuatrimestre del 2026

GRUPO N°3 — INTEGRANTES

Apellido y Nombre	Matrícula
Dominguez, Franco Agustín	67575
Hillebrand, Giuliano	67625
González, Martín Gastón	67501
Valchar, Angelo Iván	67585
Viera, Marcia	64700

Índice

Declaración de uso informado de inteligencia artificial generativa	3
Introducción	5
Modalidad de trabajo	5
Rol del docente	5
Comunicación equipo de IT y cliente	5
Descripción del escenario	6
Etapas del trabajo	7
Etapa 1: modelado de la base de datos	8
Análisis de requisitos	8
Diagrama entidad-relación	13
Consideraciones de diseño del modelo de datos	15
Casos de uso	17
Actores	17
Catálogo de casos de uso	18
Diagrama de casos de uso	19
Mapeo caso de uso ↔ modelo de datos	20
Detalle de casos de uso principales	21
<i>CU-03: Reservar vehículo online</i>	<i>21</i>
<i>CU-05: Iniciar alquiler (retiro del vehículo)</i>	<i>22</i>
<i>CU-06: Finalizar alquiler y generar factura</i>	<i>22</i>
<i>CU-07: Enviar vehículo a mantenimiento</i>	<i>23</i>
Etapa 2: implementación en el SGBD	25
Requerimientos técnicos de la Etapa 2	26
Pila tecnológica	27
Arquitectura del schema	29
Catálogo de objetos DDL	30
Implementación de los casos de uso	38
Auditoría — triple identidad	40
Excepciones y transacciones	41
Mapeo de requerimientos	42
Diagrama ER actualizado	43

Conclusiones	45
Etapas 3: presentación de la solución final	46
Objetivo y alcance de la etapa	46
Cierre de la lógica de negocio en el SGBD	46
Procesamiento masivo programado: expiración de reservas	47
Cierre contable mensual de facturación	48
La capa de interfaz: arquitectura y alcance	49
Módulos implementados	49
Recorrido funcional de los casos de uso	51
Seguridad de la capa de interfaz	52
Conclusiones	52
Referencias	52

Declaración de uso informado de inteligencia artificial generativa

Los integrantes del Grupo 3 declaran haber utilizado herramientas de inteligencia artificial generativa durante la elaboración de este documento, en el marco de un uso informado y responsable. Su empleo se circunscribió a las siguientes actividades de soporte:

- **Formato y diseño visual:** estructuración del documento (encabezados, tablas, separación de secciones), generación de los diagramas (entidad-relación en Mermaid, casos de uso en PlantUML) a partir del diccionario de datos definido por los autores, y aplicación de criterios tipográficos consistentes.
- **Asistencia de redacción:** sugerencias de fraseo, mejora de la claridad expositiva y revisión ortográfica y gramatical sobre textos previamente concebidos por los autores.
- **Revisión de contenido:** detección de inconsistencias internas entre el diccionario de datos, las relaciones, el diagrama y los casos de uso; verificación de la trazabilidad entre requisitos del enunciado y entidades del modelo.

El **análisis del problema, las decisiones de diseño y la justificación de cada elección** — incluyendo la elección de las entidades, la normalización aplicada, la introducción de catálogos (`estado_vehiculo` , `tipo_reserva`), la separación entre pertenencia y ubicación física del vehículo y el patrón de **snapshot** (*instantánea: copia congelada del estado de un dato en un momento dado*) en `factura` — son obra y responsabilidad íntegra de los autores. La inteligencia artificial actuó exclusivamente como herramienta de soporte; en ningún caso reemplazó el juicio técnico ni la autoría intelectual del grupo.

Durante la implementación de la Etapa 2, se mantuvo el mismo marco de uso responsable. La inteligencia artificial asistió en la redacción técnica, el formateo de tablas comparativas y la revisión de consistencia entre los procedimientos almacenados y los requerimientos de la cátedra. El diseño de cada función, vista y disparador, así como la justificación arquitectónica de elegir PostgreSQL + Supabase como conjunto de tecnologías y de exponer la lógica de negocio mediante `FUNCTION` s en lugar de `PROCEDURE` s, son obra y responsabilidad íntegra del grupo de autores.

Un caso particular dentro de este trabajo merece una declaración propia: la aplicación web que acompaña la entrega —descrita en la subsección Sobre la interfaz auxiliar y cuyo cometido es ofrecer las interfaces necesarias para operar y consultar el sistema— fue desarrollada en su totalidad con asistencia de inteligencia artificial generativa. El equipo optó por delegar su construcción a herramientas de IA, partiendo de los requisitos funcionales definidos previamente por los autores y de los procedimientos, funciones y vistas ya disponibles en el SGBD.

Esta delegación es legítima a la luz de la propia consigna. El TFI sitúa a esta capa como un componente complementario al SGBD, aclarando explícitamente que "no abarcará la totalidad del

sistema" (TFI 2026, sección *Etapas*, ítem 3). Su rol es exponer la lógica ya implementada en el motor, sin contener reglas de negocio propias y operando desacoplada del modelo de datos. Por su carácter complementario y por estar fuera del eje del trabajo —cuyo foco es la base de datos—, su construcción no compromete la autoría intelectual sobre la pieza que sí se evalúa.

La implicancia para este informe es directa: el comportamiento que la aplicación expone es el mismo que se documenta aquí sobre la capa de base de datos. La interfaz no agrega ni altera nada respecto de lo descrito; toda regla de negocio que el lector observe en la aplicación está definida y justificada en este documento.

Introducción

Este documento reúne la información de la consigna del Trabajo Final Integrador y funciona como documentación del proceso de diseño e implementación de la base de datos que busca resolver una problemática real. Además, este documento es de elaboración propia de los integrantes del Grupo 3.

En el marco de la materia, se desarrollará un Trabajo Final Integrador (TFI) cuyo objetivo es aplicar de manera práctica los conceptos vistos durante el cursado, abordando inicialmente el diseño y posterior implementación de una base de datos.

El trabajo se organizará en distintas etapas, permitiendo una evolución progresiva del sistema, incorporando nuevos requisitos y complejidades a lo largo del desarrollo.

Modalidad de trabajo

El Trabajo Integrador deberá realizarse en grupos de cinco (5) alumnos.

Se espera que todos los integrantes participen activamente en el análisis, diseño y desarrollo de la solución propuesta.

Rol del docente

El docente actúa como cliente a satisfacer.

Por lo tanto:

- Toda consulta sobre el entendimiento de la lógica del sistema deberá ser realizada al docente.
- La propuesta de diseño deberá ser aprobada por el cliente antes de su implementación en la base de datos.

Comunicación equipo de IT y cliente

Por medio del servidor de Discord ya establecido para la materia.

Descripción del escenario

Una empresa de alquiler de vehículos posee varias sucursales, en las cuales se dispone de autos para alquilar.

Los clientes pueden reservar para posteriormente alquilar vehículos por un período determinado, indicando la fecha de inicio y la fecha de devolución prevista.

Requisitos del sistema

- Al momento de iniciar el alquiler, se debe registrar la cantidad de kilómetros del vehículo, y al momento de finalizarlo, se debe registrar nuevamente esta información.
- Cada vehículo pertenece a una sucursal y tiene un tipo (por ejemplo: cupé, sedán, SUV, etc.), además de información descriptiva como marca, modelo y un detalle de confort.
- Por cada vehículo se podrán registrar entre 1 y 5 imágenes que permitan al cliente visualizar sus características.
- Los vehículos pueden estar en distintos estados, tales como: disponible, alquilado o en mantenimiento.
- Cada alquiler corresponde a un único cliente y a un único vehículo.
- El valor del alquiler es por día, y depende tanto del tipo de vehículo como de la sucursal donde se realiza el alquiler.
- Asimismo, en caso de que el vehículo sea devuelto fuera del tiempo pactado, se deberá cobrar un recargo por hora excedida, calculado como un porcentaje del valor diario del alquiler. Dicho porcentaje también depende del tipo de vehículo y de la sucursal.
- El sistema deberá generar una factura al finalizar el alquiler a nombre del cliente, donde se detalle el costo del alquiler y los recargos aplicados.
- Los clientes podrán realizar reservas de vehículos de forma online, para lo cual deberán registrarse en el sistema mediante un usuario y contraseña.
- La reserva de un vehículo deberá validar la disponibilidad del mismo en las fechas solicitadas, evitando superposición con otros alquileres o reservas existentes.
- El estado "en mantenimiento" del vehículo surge del envío del mismo a un taller mecánico, dicho envío debe ser registrado, y deberá incluirse la información de fecha envío y fecha de devolución.

Etapas del trabajo

1. Modelado de la Base de datos que solucione el problema (30/04/2026)
2. Implementación en el SGBD elegido, más lógica de negocio parcial aplicada en procedimientos / funciones / disparadores, etc. (21/05/2026)
3. Presentación de la solución final, lógica de negocio terminada en el SGBD más desarrollado en framework a elección, de las interfaces necesarias para el funcionamiento del sistema. (11/06/2026)

Importante: La implementación en framework no abarcará la totalidad del sistema. Los módulos a desarrollar serán definidos durante el cursado.

Etapa 1: modelado de la base de datos

Análisis de requisitos

Del enunciado se extraen los siguientes requisitos principales:

Entidades y atributos

Entidad	Atributos	Observaciones
alquiler	id_alquiler, id_reserva, id_cliente, id_vehiculo, id_tarifa, id_sucursal_devolucion, fecha_inicio, fecha_fin_prevista, fecha_devolucion_real, km_inicio, km_fin, estado.	Registra cada alquiler efectivo de un vehículo por parte de un cliente, incluyendo los kilómetros al inicio y al cierre, las fechas pactadas y la devolución real. La FK <code>id_sucursal_devolucion</code> permite registrar devoluciones cruzadas en una sucursal distinta a la de origen del vehículo.
cliente	id_cliente, id_usuario, nombre, apellido, dni, telefono, direccion.	Representa a la persona física que realiza alquileres y reservas. Puede estar vinculado a un usuario del sistema para operar de forma online.
estado_vehiculo	id_estado, nombre, descripcion.	Catálogo de estados posibles del vehículo ('disponible', 'alquilado', 'en_mantenimiento', 'en_traslado', etc.). Reemplaza al enum embebido para permitir agregar estados nuevos sin alterar la estructura de la tabla <code>vehiculo</code> .
factura	id_factura, id_alquiler, id_cliente, numero_factura, fecha_emision, precio_por_dia_aplicado, porcentaje_recargo_aplicado, costo_base, horas_excedidas, recargo_excedente, total.	Comprobante generado al finalizar un alquiler, detallando el costo base por días y los recargos por horas excedidas si correspondiera. Se conserva <code>id_cliente</code> como snapshot histórico del cliente facturado. Se almacenan además <code>precio_por_dia_aplicado</code> y <code>porcentaje_recargo_aplicado</code> como snapshot de la tarifa vigente al momento de emisión, garantizando que la factura sea autocontenida para auditoría fiscal independientemente de cambios futuros en la tabla <code>tarifa</code> .
garantia_reserva	id_garantia, id_reserva, tipo, titular, numero_tarjeta_hash, vencimiento, fecha_registro, activa.	Almacena la información de la tarjeta de crédito (u otro medio de garantía) que respalda una reserva. El campo <code>activa</code> (boolean) permite invalidar la garantía sin eliminarla cuando la reserva se cancela, preservando la trazabilidad. Tabla separada por aislamiento de datos sensibles y flexibilidad para incorporar otros tipos de garantía a futuro.

Entidad	Atributos	Observaciones
historial_estado_vehiculo	id_historial, id_vehiculo, id_estado, fecha_inicio, fecha_fin, motivo.	Registra cada transición de estado de un vehículo a lo largo del tiempo, permitiendo auditar cuándo y por qué cambió. El registro vigente tiene <code>fecha_fin</code> en NULL.
imagen_vehiculo	id_imagen, id_vehiculo, url_imagen, orden.	Almacena entre 1 y 5 imágenes asociadas a cada vehículo, permitiendo al cliente visualizar sus características antes de reservar.
mantenimiento	id_mantenimiento, id_vehiculo, id_taller, fecha_envio, fecha_devolucion, observaciones.	Registra el historial de envíos de un vehículo a un taller mecánico, incluyendo las fechas de entrada y salida y las observaciones del servicio realizado.
reserva	id_reserva, id_cliente, id_vehiculo, id_tipo_reserva, fecha_inicio, fecha_fin_prevista, estado, fecha_creacion.	Representa la reserva online de un vehículo por parte de un cliente para un período determinado, validando que no exista superposición con otros alquileres o reservas activas. La FK <code>id_tipo_reserva</code> define qué reglas de negocio (por ejemplo, obligatoriedad de garantía) aplican a la reserva.
sucursal	id_sucursal, nombre, direccion, ciudad, telefono.	Cada sede física de la empresa donde se encuentran los vehículos disponibles para alquilar y donde se gestionan los alquileres presenciales.
taller	id_taller, nombre, direccion, telefono.	Taller mecánico externo al que se envían los vehículos cuando requieren mantenimiento, dejando el vehículo fuera de disponibilidad durante ese período.
tarifa	id_tarifa, id_sucursal, id_tipo, precio_por_dia, porcentaje_recargo.	Define el precio diario del alquiler y el porcentaje de recargo por hora excedida según la combinación de tipo de vehículo y sucursal. <code>porcentaje_recargo</code> se almacena como decimal en formato fracción (por ejemplo, 0.20 representa 20%). Cada combinación (id_sucursal, id_tipo) es única.
tipo_reserva	id_tipo_reserva, nombre, descripcion, requiere_garantia, antelacion_max_dias.	Catálogo de tipos de reserva permitidos (estándar, express, corporativa, etc.). Permite condicionar reglas de negocio diferenciales como obligatoriedad de garantía y plazos máximos de antelación, sin codificarlas en la lógica de aplicación.
tipo_vehiculo	id_tipo, nombre, descripcion.	Categoría que clasifica a los vehículos según su carrocería (Sedán, SUV, Cupé, etc.), siendo un factor determinante en el cálculo de la tarifa.

Entidad	Atributos	Observaciones
ubicacion_vehiculo	id_ubicacion, id_vehiculo, id_sucursal, fecha_desde, fecha_hasta.	Registra en qué sucursal se encuentra físicamente cada vehículo en cada momento. Disocia la pertenencia (sucursal de origen, atributo de vehiculo) de la ubicación operativa (esta tabla). El registro vigente tiene fecha_hasta en NULL.
usuario	id_usuario, username, password_hash, email, created_at.	Credenciales de acceso al sistema online para que los clientes puedan realizar reservas de forma remota mediante usuario y contraseña.
vehiculo	id_vehiculo, id_sucursal_origen, id_tipo, id_estado, marca, modelo, anio, patente, km_actuales, detalle_confort.	Unidad disponible para alquilar, perteneciente (origen) a una sucursal y clasificada por tipo. Su estado actual se referencia mediante id_estado (FK al catálogo estado_vehiculo) y su ubicación física se gestiona en la tabla ubicacion_vehiculo .

Relaciones

Relación	Cardinalidad	Descripción
alquiler a factura	1:1	Factura generada por cada alquiler finalizado
cliente a alquiler	1:N	Un cliente puede realizar múltiples alquileres
cliente a reserva	1:N	Un cliente puede realizar múltiples reservas
cliente a usuario	1:0..1	Un cliente puede tener un usuario para el inicio de sesión (no obligatorio: los clientes presenciales no requieren cuenta online)
estado_vehiculo a vehiculo	1:N	Cada vehículo referencia un estado del catálogo
estado_vehiculo a historial_estado_vehiculo	1:N	Cada estado puede aparecer en múltiples transiciones registradas

Relación	Cardinalidad	Descripción
reserva a alquiler	0..1:0..1	Una reserva puede concretarse en un alquiler (no obligatorio); un alquiler puede provenir de una reserva o ser presencial sin reserva previa
reserva a garantia_reserva	1:0..1	Una reserva tiene a lo sumo una garantía asociada (obligatoria solo si el tipo de reserva la exige)
sucursal a alquiler (devolución)	1:N	Sucursal donde se devuelve el vehículo (puede diferir de la de origen)
sucursal a tarifa	1:N	Una sucursal define múltiples tarifas según el tipo de vehículo
sucursal a ubicacion_vehiculo	1:N	Una sucursal aloja múltiples vehículos a lo largo del tiempo
sucursal a vehiculo (origen)	1:N	Cada vehículo pertenece (origen) a una sucursal
taller a mantenimiento	1:N	Un taller recibe múltiples envíos de mantenimiento
tarifa (tipo_vehiculo y sucursal) a alquiler	1:N	Precios definidos según tipo de vehículo y sucursal
tipo_reserva a reserva	1:N	Cada reserva pertenece a un tipo definido en el catálogo
tipo_vehiculo a tarifa	1:N	Cada tipo de vehículo participa en múltiples tarifas según la sucursal
tipo_vehiculo a vehiculo	1:N	Clasificación de vehículos por tipo
vehiculo a alquiler	1:N	Un vehículo puede tener múltiples alquileres
vehiculo a historial_estado_vehiculo	1:N	Histórico de transiciones de estado de un vehículo

Relación	Cardinalidad	Descripción
vehiculo a imagen_vehiculo	1:1..5	Entre 1 y 5 imágenes obligatorias por vehículo (cota inferior reforzada por aplicación o trigger; cota superior por trigger sobre <code>imagen_vehiculo</code>)
vehiculo a mantenimiento	1:N	Histórico de mantenimientos de un vehículo
vehiculo a reserva	1:N	Un vehículo puede tener múltiples reservas
vehiculo a ubicacion_vehiculo	1:N	Histórico de ubicaciones físicas de un vehículo

Restricciones adicionales de unicidad

A continuación se listan las restricciones `UNIQUE` que el modelo asume y que se aplicarán al implementar el esquema en el SGBD (Etapa 2). Estas no se reflejan en la notación Mermaid utilizada para el DER porque el dialecto no incluye un símbolo nativo para unicidad fuera de las claves primarias.

Tabla	Columna(s) <code>UNIQUE</code>	Justificación
<code>usuario</code>	<code>username</code>	Identificador único de acceso.
<code>usuario</code>	<code>email</code>	Recuperación de cuenta y canal de contacto.
<code>cliente</code>	<code>dni</code>	Identificador legal único del titular.
<code>cliente</code>	<code>id_usuario</code>	Refuerza la cardinalidad 1:0..1 con <code>usuario</code> .
<code>vehiculo</code>	<code>patente</code>	La patente identifica unívocamente al vehículo en la realidad.
<code>tarifa</code>	<code>(id_sucursal, id_tipo)</code>	Una sola tarifa por combinación de sucursal y tipo de vehículo.
<code>factura</code>	<code>numero_factura</code>	Numeración correlativa única exigida por auditoría fiscal.

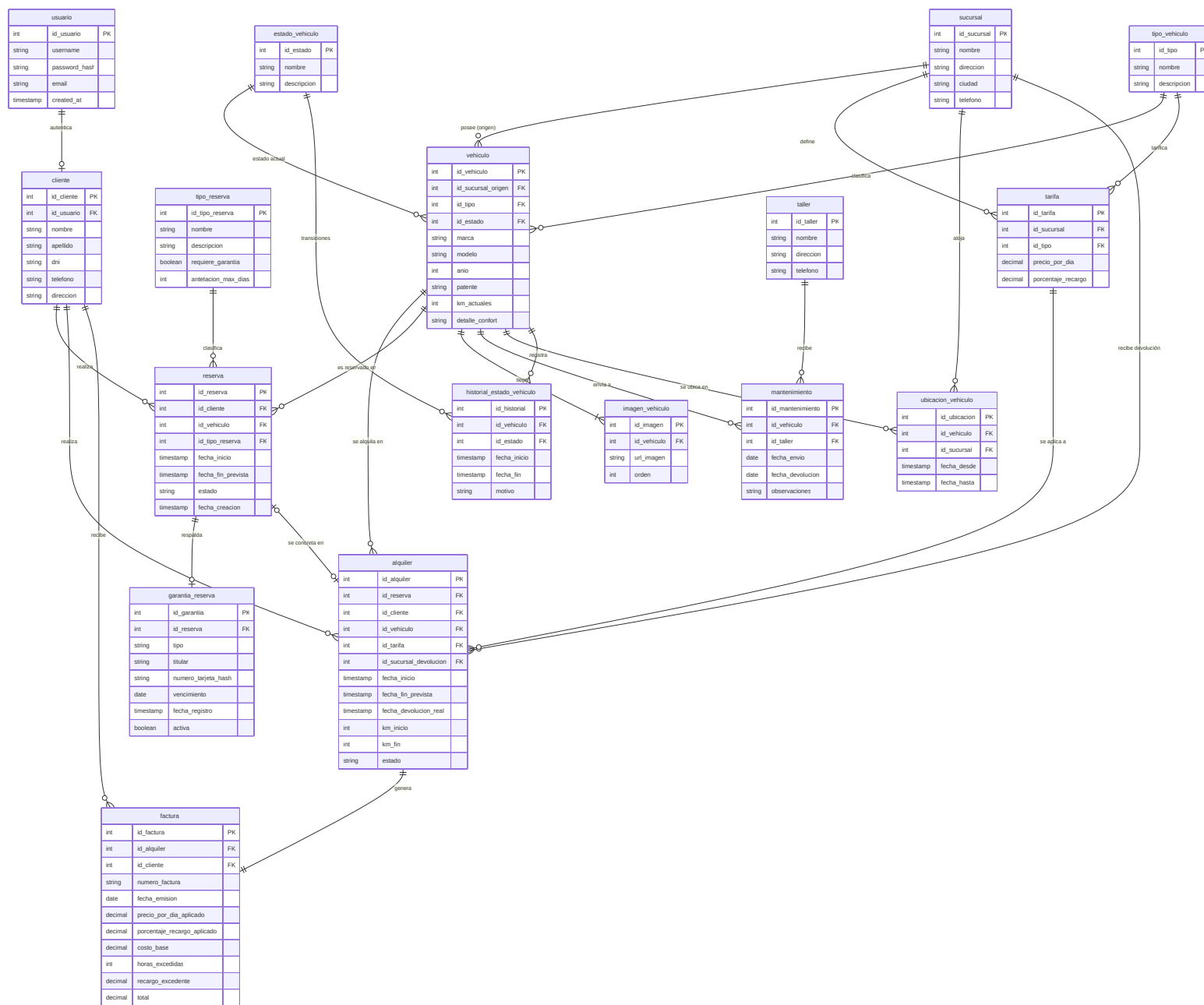
Tabla	Columna(s) UNIQUE	Justificación
<code>imagen_vehiculo</code>	<code>(id_vehiculo, orden)</code>	Garantiza un único registro por posición de imagen en el vehículo.

Para las tablas que mantienen un único registro vigente con `fecha_hasta IS NULL` (`ubicacion_vehiculo`, `historial_estado_vehiculo`), se sugiere un **índice parcial único** sobre `id_vehiculo` filtrado por `fecha_hasta IS NULL` (o `fecha_fin IS NULL`, según corresponda) para impedir dos registros activos simultáneos del mismo vehículo. La sintaxis exacta dependerá del SGBD elegido en la Etapa 2.

Diagrama entidad-relación

El modelo de datos que soluciona –o que busca solucionar– los problemas del escenario del TFI 2026 fue modelado en una primera instancia utilizando dbdiagram.io, una herramienta que toma DBML (Database Markup Language) y lo modela gráficamente, tomando como referencia conceptual las notaciones descritas en el *Appendix A: Alternative Diagrammatic Notations for ER Models* del libro de Elmasri & Navathe [1, p. 1164]. Para la versión final embebida en este documento se optó por la notación **Crow's Foot** renderizada con Mermaid, dado que es la que mejor se adapta a la representación textual versionable y a la generación automática del diagrama desde el diccionario de datos. Esta notación expresa la cardinalidad de cada extremo de la relación mediante símbolos de "pata de cuervo", siendo equivalente en expresividad a las notaciones (b)(iii) y (d)(iv) del Apéndice citado.

A continuación se presenta el DER renderizado mediante notación Mermaid (Crow's Foot), generado a partir del diccionario de datos detallado en la sección anterior.



Notación de cardinalidades (Mermaid Crow's Foot):

Símbolo	Significado
	exactamente uno (obligatorio)
o	cero o uno (opcional)
{	uno o muchos (obligatorio)
o{	cero o muchos (opcional)

Consideraciones de diseño del modelo de datos

- **Tabla `tarifa`** : resuelve la doble dependencia del precio. El enunciado exige que `precio_por_dia` y `porcentaje_recargo` dependan tanto del tipo de vehículo como de la sucursal. Esto descarta colocarlos en `vehiculo` (provocaría desnormalización masiva, repitiendo el precio en cada vehículo del mismo tipo y sucursal), en `tipo_vehiculo` (pierde la dependencia con sucursal) o en `sucursal` (pierde la dependencia con tipo). La solución normalizada es esta tabla intermedia con la restricción `UNIQUE(id_sucursal, id_tipo)` , que asegura una y solo una tarifa por combinación.
- **Separación `usuario` / `cliente`** : `usuario` maneja las credenciales (username/password) para el acceso online, mientras que `cliente` tiene los datos personales. Así el sistema puede tener empleados u otros usuarios en el futuro sin mezclar responsabilidades.
- **Reserva y alquiler**: la reserva online es un paso previo; cuando el cliente se presenta a retirar el auto, se genera el alquiler concreto (con kilómetros de inicio, tarifa efectiva, etc.). La relación es opcional porque no toda reserva necesariamente se concreta.
- **Tabla `imagen_vehiculo`** : tabla separada con un campo `orden` (1 a 5) para cumplir la restricción de entre 1 y 5 imágenes por vehículo. La validación del mínimo de 1 puede hacerse a nivel de disparador o de aplicación.
- **Mantenimiento**: registra el envío al taller con fechas de envío y devolución, y una FK hacia `taller` para tener información del mecánico. Al insertar un registro aquí, un disparador inserta una nueva fila en `historial_estado_vehiculo` con el estado 'en_mantenimiento' y propaga el cambio a `vehiculo.id_estado` . Al registrar la devolución, otro disparador realiza la transición inversa al estado 'disponible'.
- **Factura con snapshot de tarifa**: `factura` almacena `id_cliente` como FK directa (snapshot del cliente facturado al momento de emisión, para trazabilidad fiscal aunque el alquiler se modifique), y adicionalmente `precio_por_dia_aplicado` y `porcentaje_recargo_aplicado` como snapshot de la tarifa vigente al cierre del alquiler. Esto garantiza que la factura sea autocontenida: si los valores de `tarifa` se actualizan en el

futuro, el documento fiscal refleja exactamente las condiciones pactadas sin depender de que la tabla `tarifa` sea inmutable.

- **Estados textuales en `reserva` y `alquiler`**: a diferencia del estado de `vehiculo` — modelado como catálogo `estado_vehiculo` con su tabla `historial_estado_vehiculo` — los estados de `reserva` y `alquiler` se mantienen como columnas `string` con dominio cerrado: `reserva.estado ∈ {'pendiente', 'concretada', 'cancelada'}` y `alquiler.estado ∈ {'activo', 'cerrado'}`. Se descarta la tabla catálogo porque (1) el conjunto de valores es estable y no se prevé crecimiento, (2) no se requiere asociar atributos adicionales a cada estado —a diferencia de `estado_vehiculo`, donde podrían sumarse reglas de transición o etiquetas operativas— y (3) la trazabilidad temporal queda cubierta por las propias marcas de tiempo de las filas (`reserva.fecha_creacion`, `alquiler.fecha_inicio`, `alquiler.fecha_devolucion_real`), sin necesidad de un historial adicional.
- **Tipos de fecha (`timestamp` vs `date`)**: los campos `fecha_inicio`, `fecha_fin_prevista` y `fecha_devolucion_real` de `alquiler` —y por consistencia los homólogos en `reserva` — se modelan como `timestamp` (no como `date`), dado que el cálculo del recargo por horas excedidas exigido por el enunciado requiere componente horaria. Los campos puramente diarios (`mantenimiento.fecha_envio`, `mantenimiento.fecha_devolucion`, `factura.fecha_emision`) se mantienen como `date`.

Justificación de los cambios incorporados a partir de la retroalimentación del cliente

El cliente solicitó que se justificara explícitamente el porqué de las siguientes decisiones de diseño. Cada una responde a un requisito o restricción del escenario que no podía cubrirse adecuadamente con el modelo previo.

- **Estado del vehículo como tabla (`estado_vehiculo` + `historial_estado_vehiculo`)**: en la versión inicial el estado era un campo enumerado embebido en `vehiculo`. Se decidió migrar a una tabla catálogo separada porque (1) el conjunto de estados puede crecer (por ejemplo, 'en_traslado' para movimientos entre sucursales) sin requerir cambios estructurales, (2) habilita relaciones referenciales que un enum no permite (por ejemplo, asociar reglas de transición a cada estado) y (3) la tabla `historial_estado_vehiculo` agrega trazabilidad temporal: permite reconstruir el ciclo de vida operativo de cualquier vehículo, dato exigido por la auditoría interna en escenarios reales de la industria del rent-a-car.
- **Ubicación del vehículo como tabla (`ubicacion_vehiculo`)**: el escenario contempla la devolución de un vehículo en una sucursal distinta a la de origen, pero el cliente especificó que el vehículo "sigue perteneciendo a la sucursal origen" para no complicar la operatoria. Esto introduce una distinción conceptual fundamental: la **pertenencia** (atributo `id_sucursal_origen` de `vehiculo`, inmutable, base para el cálculo de tarifa) es distinta de la **ubicación física** (sucursal donde el vehículo se encuentra disponible para retiro en un momento dado). Un campo simple en `vehiculo` no bastaría: se necesita historial para auditar

el flujo físico entre sucursales y para resolver la disponibilidad real al consultar el catálogo. La tabla `ubicacion_vehiculo` resuelve ambos requisitos.

- **Tipos de reserva (`tipo_reserva`) y garantía con tarjeta (`garantia_reserva`)**: el cliente confirmó que las reservas no requieren seña monetaria sino una tarjeta de crédito como garantía. Para no codificar esta regla en la lógica de aplicación —y para anticipar la posibilidad de tipos de reserva con reglas distintas (express, corporativa, etc.)— se introduce el catálogo `tipo_reserva` , donde cada tipo declara si requiere garantía y qué antelación máxima admite. La garantía se modela en una tabla separada (`garantia_reserva`) por dos motivos: (1) los datos de tarjeta son sensibles y conviene aislarlos para aplicar políticas de acceso y cifrado específicas, y (2) deja abierta la posibilidad de incorporar otros tipos de garantía (depósito en efectivo, voucher corporativo) sin migrar columnas en `reserva` .
- **Devolución cruzada (`alquiler.id_sucursal_devolucion`)**: se agrega esta FK opcional para registrar la sucursal donde el cliente efectivamente entregó el vehículo. Cuando difiere de `vehiculo.id_sucursal_origen` , los disparadores de finalización del alquiler actualizan `ubicacion_vehiculo` para reflejar la nueva ubicación física, sin alterar la pertenencia. La tarifa aplicada al alquiler sigue siendo la de la sucursal de origen, evitando complejidades adicionales no solicitadas por el cliente.

Casos de uso

El diagrama entidad-relación describe la **estructura** del sistema: qué información se almacena y cómo se relaciona. Los casos de uso describen el **comportamiento**: cómo los actores interactúan con esa información para satisfacer los requisitos del enunciado. Ambas vistas se sostienen sobre el mismo diccionario de datos definido previamente, garantizando consistencia entre lo que el sistema guarda y lo que permite hacer.

Esta sección sirve además como puente hacia la Etapa 2: cada caso de uso aterrizará en procedimientos almacenados, funciones y disparadores concretos sobre las entidades del modelo.

Actores

Actor	Descripción
Cliente anónimo	Visitante que consulta el catálogo sin estar autenticado.
Cliente registrado	Persona física con usuario y contraseña que opera de forma online (reservas, cancelaciones, consultas).

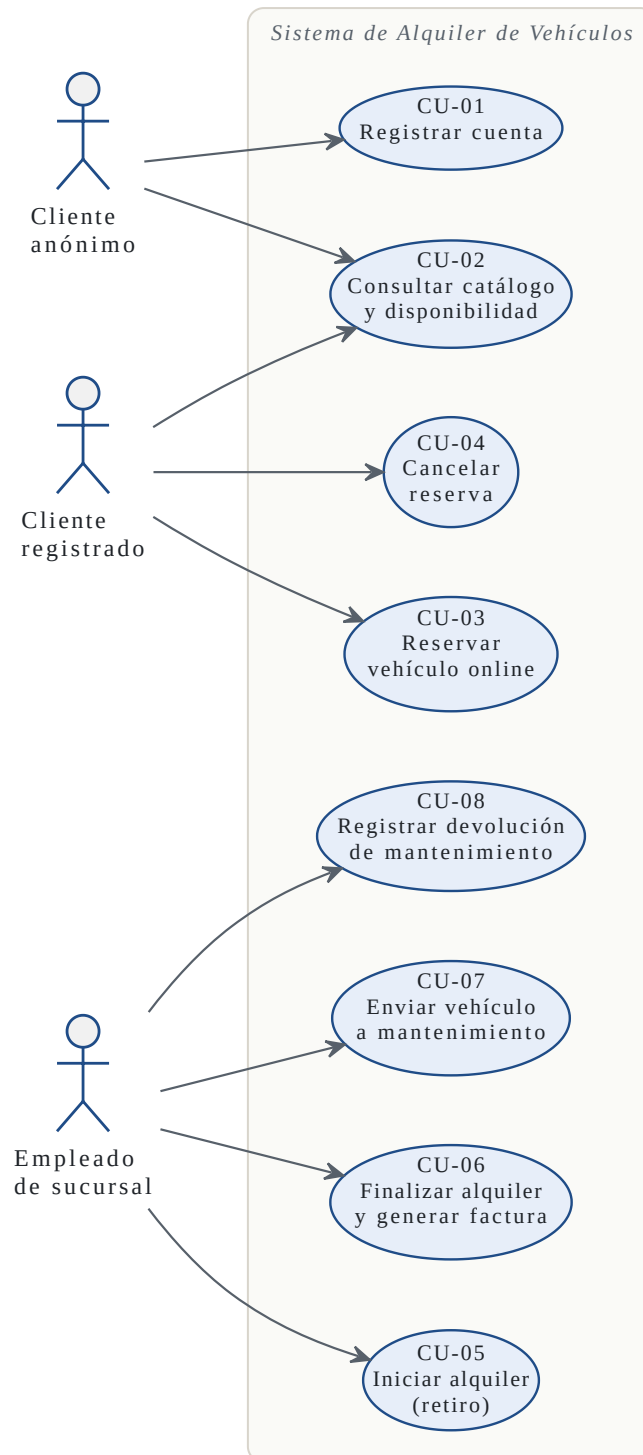
Actor	Descripción
Empleado de sucursal	Personal de la empresa que gestiona el ciclo presencial: retiros, devoluciones, envíos a taller.
Sistema	Actor implícito que ejecuta lógica automática mediante disparadores (cambios de estado, validaciones de superposición, cálculo de totales).

Catálogo de casos de uso

ID	Nombre	Actor principal
CU-01	Registrar cuenta de cliente	Cliente anónimo
CU-02	Consultar catálogo y disponibilidad	Cliente anónimo / registrado
CU-03	Reservar vehículo online	Cliente registrado
CU-04	Cancelar reserva	Cliente registrado
CU-05	Iniciar alquiler (retiro del vehículo)	Empleado de sucursal
CU-06	Finalizar alquiler y generar factura	Empleado de sucursal
CU-07	Enviar vehículo a mantenimiento	Empleado de sucursal
CU-08	Registrar devolución de mantenimiento	Empleado de sucursal

Diagrama de casos de uso

El siguiente diagrama representa visualmente la relación entre actores y casos de uso, delimitando la frontera del sistema. El actor *Sistema* no se representa explícitamente: ejecuta lógica automática (disparadores, validaciones, cálculos) que se dispara como consecuencia de los casos de uso de uso iniciados por actores humanos.



Mapeo caso de uso ↔ modelo de datos

Cada caso de uso opera sobre un subconjunto del diccionario de datos. La columna *Operaciones* indica el tipo de acceso (C=Create, R=Read, U=Update) sobre cada entidad.

ID	Caso de uso	Entidades involucradas	Operaciones	Lógica de negocio crítica
CU-01	Registrar cuenta de cliente	usuario, cliente	C usuario, C cliente	Unicidad de username, email y dni. Hash de password. Vinculación 1:1 usuario ↔ cliente.
CU-02	Consultar catálogo y disponibilidad	vehiculo, imagen_vehiculo, tipo_vehiculo, sucursal, tarifa, ubicacion_vehiculo, estado_vehiculo	R todas	Filtros por sucursal/tipo/fechas. Resolución de tarifa por (sucursal, tipo). Solo vehiculos cuyo <code>id_estado</code> corresponde a 'disponible' y cuya ubicación vigente coincide con la sucursal de retiro deseada.
CU-03	Reservar vehículo online	reserva, tipo_reserva, garantia_reserva, vehiculo, cliente	C reserva, C garantia_reserva, R tipo_reserva, R vehiculo, R cliente	Validación de superposición contra otras reservas y alquileres activos del mismo vehiculo. Si <code>tipo_reserva.requiere_garantia = true</code> , se exige carga de tarjeta en <code>garantia_reserva</code> . estado inicial='pendiente'.
CU-04	Cancelar reserva	reserva, garantia_reserva	U reserva, U garantia_reserva	Cambio de estado a 'cancelada'. Solo reservas en estado 'pendiente'. Libera disponibilidad. La garantía asociada (si existía) se marca como inactiva (no se elimina, para preservar trazabilidad).
CU-05	Iniciar alquiler (retiro del vehículo)	alquiler, reserva, vehiculo, tarifa, cliente, ubicacion_vehiculo, historial_estado_vehiculo	C alquiler, U reserva, U vehiculo, C historial_estado_vehiculo, R tarifa, R cliente, R ubicacion_vehiculo	Registro de km_inicio. Validación de que la ubicación física vigente del vehículo coincide con la sucursal de retiro. Resolución de <code>id_tarifa</code> por (sucursal_origen, tipo). Si proviene de reserva, marcarla como 'concretada'. Trigger: cambio de estado a 'alquilado' (registrado en <code>historial_estado_vehiculo</code> y propagado a <code>vehiculo.id_estado</code>).

ID	Caso de uso	Entidades involucradas	Operaciones	Lógica de negocio crítica
CU-06	Finalizar alquiler y generar factura	alquiler, factura, vehiculo, tarifa, ubicacion_vehiculo, historial_estado_vehiculo	U alquiler, C factura, U vehiculo, C historial_estado_vehiculo, U ubicacion_vehiculo (cierra registro previo, abre nuevo en sucursal de devolución), R tarifa	Registro de km_fin, fecha_devolucion_real y id_sucursal_devolucion. Cálculo de días, horas_excedidas, costo_base, recargo_excedente y total. Snapshot de precio_por_dia y porcentaje_recargo copiados desde tarifa en el momento de emisión. Triggers: cambio de estado a 'disponible' y actualización de ubicacion_vehiculo con la sucursal de devolución (que puede diferir de la de origen).
CU-07	Enviar vehículo a mantenimiento	mantenimiento, vehiculo, taller, historial_estado_vehiculo	C mantenimiento, U vehiculo, C historial_estado_vehiculo, R taller	Validación de que vehiculo.id_estado es 'disponible'. Trigger: cambio de estado a 'en_mantenimiento'. La ubicación física no se modifica (el vehículo sigue asociado a la sucursal donde estaba).
CU-08	Registrar devolución de mantenimiento	mantenimiento, vehiculo, historial_estado_vehiculo	U mantenimiento, U vehiculo, C historial_estado_vehiculo	Registro de fecha_devolucion y observaciones. Trigger: cambio de estado a 'disponible'.

Detalle de casos de uso principales

Se desarrollan a continuación los casos de uso de mayor complejidad, dado que concentran la lógica de negocio que se materializará en procedimientos y disparadores en la Etapa 2.

CU-03: Reservar vehículo online

- **Actor:** Cliente registrado.
- **Precondición:** el cliente está autenticado mediante usuario y contraseña.
- **Flujo principal:**
 1. El cliente selecciona un vehículo del catálogo, indica fecha de inicio, fecha de fin prevista y elige el tipo de reserva.
 2. El sistema valida que el id_estado del vehículo corresponde a 'disponible'.
 3. El sistema valida que no exista superposición de fechas con alquileres en curso ni con reservas activas (en estado 'pendiente' o 'concretada') del mismo vehículo.

4. Si `tipo_reserva.requiere_garantia = true`, el sistema solicita los datos de la tarjeta de crédito (titular, número, vencimiento). El número se almacena **hasheado** (*sometido a una transformación irreversible que guarda una huella del dato sin conservar el dato original*).
 5. El sistema crea un registro en `reserva` con `estado='pendiente'` y `fecha_creacion=NOW()`.
 6. Si aplica, el sistema crea el registro en `garantia_reserva` vinculado a la reserva.
- **Postcondición:** la reserva queda registrada y bloquea la disponibilidad del vehículo en el rango solicitado. Si el tipo lo requiera, la garantía queda asociada.
 - **Flujo alternativo:** si existe superposición o si el tipo de reserva exige garantía y el cliente no la provee, se rechaza la reserva.

CU-05: Iniciar alquiler (retiro del vehículo)

- **Actor:** Empleado de sucursal.
- **Precondición:** el cliente se presenta en la sucursal; opcionalmente con una reserva previa. El vehículo se encuentra físicamente en esa sucursal.
- **Flujo principal:**
 1. El empleado identifica al cliente (DNI o id_reserva).
 2. Si existe reserva, el sistema la recupera; en caso contrario, se trata de un alquiler presencial sin reserva previa.
 3. El sistema valida que la ubicación física vigente del vehículo (`ubicacion_vehiculo` con `fecha_hasta IS NULL`) coincide con la sucursal de retiro.
 4. El empleado registra los kilómetros actuales del vehículo en `alquiler.km_inicio`. Por simetría con la lectura de cierre, `vehiculo.km_actuales` no se modifica en este paso: se actualizará al finalizar el alquiler en CU-06 con el valor de `km_fin`.
 5. El sistema resuelve la tarifa aplicable mediante `tarifa(id_sucursal_origen, id_tipo)` correspondiente al vehículo. La tarifa siempre se calcula contra la sucursal de origen, sin importar dónde se retire.
 6. El sistema crea un registro en `alquiler` referenciando reserva (si aplica), cliente, vehículo y tarifa.
 7. Un disparador inserta una nueva fila en `historial_estado_vehiculo` (`estado='alquilado'`) y propaga el cambio a `vehiculo.id_estado`. Si corresponde, marca `reserva.estado='concretada'`.
- **Postcondición:** el alquiler queda activo y el vehículo bloqueado para nuevas reservas.

CU-06: Finalizar alquiler y generar factura

- **Actor:** Empleado de sucursal.
- **Precondición:** existe un alquiler en curso para el vehículo devuelto. La devolución puede realizarse en cualquier sucursal de la empresa (no necesariamente la de origen).

- **Flujo principal:**

1. El empleado registra `km_fin` , `fecha_devolucion_real` e `id_sucursal_devolucion` .
2. El sistema calcula:
 - **Días alquilados** a partir de `fecha_inicio` y `fecha_fin_prevista` . Se cobran los días pactados independientemente de que el cliente devuelva el vehículo antes de la fecha de fin prevista (política comercial estándar del rubro rent-a-car).
 - **Horas excedidas:** diferencia positiva entre `fecha_devolucion_real` y `fecha_fin_prevista` , expresada en horas.
 - **Costo base:** `dias × tarifa.precio_por_dia` .
 - **Recargo excedente:** `horas_excedidas × (tarifa.precio_por_dia / 24) × tarifa.porcentaje_recargo` . Donde `porcentaje_recargo` es decimal en formato fracción (por ejemplo, 0.20 para 20%).
 - **Total:** `costo_base + recargo_excedente` .
3. El sistema crea un registro en `factura` con número correlativo, fecha de emisión, los importes calculados y los valores de `precio_por_dia` y `porcentaje_recargo` copiados desde `tarifa` como snapshot histórico.
4. Disparadores:
 - Cierran el registro vigente de `ubicacion_vehiculo` (`fecha_hasta=NOW()`) e insertan uno nuevo con `id_sucursal=id_sucursal_devolucion` .
 - Insertan una fila en `historial_estado_vehiculo` (estado='disponible') y propagan el cambio a `vehiculo.id_estado` .
 - Actualizan `vehiculo.km_actuales=km_fin` .

- **Postcondición:** el alquiler queda cerrado, la factura emitida, el vehículo nuevamente disponible y su ubicación física registrada en la sucursal de devolución. La pertenencia (`vehiculo.id_sucursal_origen`) permanece inmutable.

CU-07: Enviar vehículo a mantenimiento

- **Actor:** Empleado de sucursal.
- **Precondición:** el vehículo está en estado 'disponible' (no puede estar 'alquilado' ni 'en_mantenimiento').
- **Flujo principal:**
 1. El empleado selecciona el vehículo y el taller destino.
 2. Registra `fecha_envio` y observaciones del motivo del servicio.
 3. El sistema crea un registro en `mantenimiento` .
 4. Un disparador inserta una nueva fila en `historial_estado_vehiculo` con el estado 'en_mantenimiento' (cerrando el registro vigente con `fecha_fin=NOW()`) y propaga el cambio a `vehiculo.id_estado` .

- **Postcondición:** el vehículo queda fuera de disponibilidad hasta que se registre su devolución (CU-08). Su ubicación física en `ubicacion_vehiculo` no se modifica: el vehículo sigue asociado a la sucursal donde estaba antes del envío al taller.

Etapa 2: implementación en el SGBD

La Etapa 2 traduce el modelo conceptual de la Etapa 1 a **DDL** (*Data Definition Language: sentencias que crean o alteran la estructura de la base —tablas, vistas, índices*) ejecutable y a la lógica de negocio que sostiene los casos de uso identificados. El motor elegido es **PostgreSQL 17.6** sobre la plataforma **BaaS** (*Backend-as-a-Service*) **Supabase**.

Antes de profundizar conviene fijar un término que recorrerá toda la sección. En este informe se usará **schema** para referirse al *conjunto declarativo de objetos de la base —tablas, vistas, funciones, disparadores, índices y permisos— que define su estructura*. En PostgreSQL la palabra también nombra un **namespace** lógico dentro de la base (`public` , `auth` , etc.); cuando aparezca en ese segundo sentido se indicará explícitamente. Los archivos `.sql` que acompañan esta entrega, agrupados en módulos numerados, materializan ese conjunto cuando se aplican en orden.

La motivación de esta combinación se explica en dos planos.

El primero es el motor en sí. PostgreSQL es el SGBD relacional de código abierto más completo del mercado, y lo elegimos por sus extensiones nativas: `btree_gist` , `pg_cron` y `pgcrypto` . Con ellas resolvemos dentro del propio motor —sin dependencias externas— las restricciones temporales, las tareas programadas y el *hashing* de datos sensibles.

El segundo plano es Supabase, que no reemplaza a PostgreSQL sino que se monta **encima del mismo motor** y le suma un conjunto de servicios listos para usar. Son cuatro: una API REST autogenerada (**PostgREST**, *API REST que PostgreSQL expone automáticamente a partir del schema, sin requerir capa de servicios escrita a mano*); un servicio de autenticación provisto por GoTrue, que emite y valida tokens **JWT** (*JSON Web Token: token firmado que transporta la identidad y los claims del usuario entre el cliente y el servidor*); políticas declarativas de seguridad a nivel de fila (**RLS**, *Row-Level Security: filtrado de filas a nivel del motor, donde cada `SELECT` aplica una política que decide qué filas puede ver el usuario actual, sin pasar por código de aplicación*); y un esquema `auth` preconfigurado donde residen esas credenciales.

La combinación es valiosa precisamente porque ambas piezas comparten el motor. La lógica de negocio puede quedarse en la base de datos —tal como exige la consigna— y, al mismo tiempo, cualquier cliente HTTP (incluida la interfaz auxiliar de consulta) invoca las funciones almacenadas como llamadas **RPC** (*Remote Procedure Call: llamada a una función almacenada de la base como si fuera un **endpoint** —punto de acceso, una URL de la API— HTTP*) sobre JSON, sin que el equipo tenga que escribir una capa de servicios intermedia.

Requerimientos técnicos de la Etapa 2

La cátedra entregó al equipo un listado de requerimientos técnicos a cumplir durante esta etapa. Se transcriben a continuación para que el lector pueda contrastar cada decisión de implementación con la consigna original:

- **R1.** Implementación de mecanismos de auditoría mediante triggers, registrando las operaciones realizadas sobre las tablas principales del sistema. Los logs deberán almacenar: usuario que realizó la operación, fecha y hora, tipo de operación (INSERT, UPDATE o DELETE). Asimismo: para operaciones INSERT deberán almacenarse los valores nuevos, para operaciones DELETE deberán almacenarse los valores anteriores, para operaciones UPDATE deberán almacenarse tanto los valores anteriores como los nuevos. La auditoría podrá implementarse mediante: una tabla de log general para todo el sistema, o tablas de log específicas por cada entidad auditada. Realizar una interfaz desde el sistema a desarrollar que permita la consulta de estos logs.
- **R2.** Todos los procedimientos almacenados deberán implementar manejo de excepciones, contemplando: control de errores, mensajes de retorno, aplicación de COMMIT y ROLLBACK en aquellos procesos que involucren transacciones.
- **R3.** Desarrollo de operaciones CRUD (Create, Read, Update, Delete) para las entidades principales del sistema. Dichas operaciones deberán implementarse mediante programación en base de datos y trabajar obligatoriamente con datos previamente cargados en el sistema.
- **R4.** Toda la lógica de inserción, actualización y eliminación de información deberá desarrollarse mediante PL/SQL y/o procedimientos almacenados equivalentes según el SGBD seleccionado. Los procedimientos deberán retornar información indicando: éxito de la operación, errores producidos, o validaciones de negocio no cumplidas.
- **R5.** Implementación y utilización de parámetros: IN, OUT, e IN OUT, en los distintos procedimientos almacenados desarrollados.
- **R6.** El sistema deberá permitir procesar alquileres: con reserva previa, o directamente sin reserva. Ambas modalidades deberán ser contempladas y correctamente validadas.
- **R7.** Desarrollo de un procedimiento almacenado encargado de registrar reservas, validando previamente: disponibilidad del vehículo, superposición de fechas, y demás restricciones necesarias. El mismo criterio deberá aplicarse al momento de registrar alquileres. Se valorará especialmente: modularización, reutilización de código, separación de responsabilidades.
- **R8.** Implementación de funcionalidad de cancelación/baja de reservas. La solución deberá contemplar las validaciones necesarias según el estado de la reserva.
- **R9.** Desarrollo de jobs/tareas programadas que se ejecuten automáticamente en horarios definidos, con el objetivo de detectar vehículos cuya fecha prevista de devolución haya expirado y aún no hayan sido entregados. La información detectada deberá almacenarse en estructuras específicas de auditoría/reportes históricos diseñadas para tal fin.
- **R10.** Implementación del proceso de finalización de alquiler, el cual deberá: registrar la devolución del vehículo, actualizar automáticamente el estado del mismo, calcular recargos

correspondientes, y generar la factura asociada al alquiler. Parte de esta lógica deberá resolverse mediante triggers y/o programación en base de datos.

Adicionalmente, durante la implementación el equipo introdujo una decisión arquitectónica propia que se documenta en este informe como **R11** (no figura en el enunciado de la cátedra).

- **R11.** Convertir los procedimientos almacenados a `FUNCTION RETURNS RECORD` para poder exponerlos vía PostgREST. La justificación se desarrolla en la subsección correspondiente al catálogo de objetos DDL.

Pila tecnológica

Capa	Tecnología	Justificación
SGBD	PostgreSQL 17.6	Extensiones nativas (<code>btree_gist</code> , <code>pg_cron</code> , <code>pgcrypto</code>) y tipos <code>tsrange</code> cubren los requisitos de tareas programadas, <i>hashing</i> y restricciones temporales sin dependencias externas. La alternativa Oracle exigía licenciamiento; MySQL no ofrece <code>EXCLUDE</code> ni <code>pg_cron</code> .
API	PostgREST 12 (Supabase)	Expone automáticamente cada <code>FUNCTION</code> declarada como endpoint <code>/rest/v1/rpc/{nombre}</code> , eliminando la necesidad de escribir capa de servicios. Cualquier cliente invoca la lógica almacenada directamente vía JSON.
Auth	GoTrue (Supabase Auth)	Emisión y validación de JWT con claves del proyecto. Un trigger sobre <code>auth.users</code> (<code>fn_handle_new_auth_user</code>) crea automáticamente el registro paralelo en <code>cliente</code> , manteniendo el modelo Etapa 1 sin acoplarse al esquema interno de Supabase.

Capa	Tecnología	Justificación
Seguridad	RLS + roles Postgres	Política <code>USING(FALSE)</code> sobre <code>audit_log</code> para roles <code>authenticated</code> / <code>anon</code> ; políticas específicas por tabla para que cada cliente solo vea sus propios alquileres. Trigger <code>BEFORE UPDATE OR DELETE</code> como segunda línea de defensa para roles con <code>BYPASSRLS</code> (atributo de rol que el motor exime de aplicar las políticas RLS).
Interfaz auxiliar	Next.js 14 + TypeScript + Tailwind CSS + <code>@supabase/ssr</code>	Cliente web que invoca las funciones almacenadas vía PostgREST y cumple el segundo párrafo de R1 (interfaz de consulta del log de auditoría). Es un complemento al schema, no el sujeto de la entrega.

A continuación se describe brevemente esta interfaz, sin entrar en detalles de implementación que escapan al alcance del informe.

Sobre la interfaz auxiliar

El grupo desarrolló una aplicación web en **Next.js 14** (framework React con renderizado en servidor) con **TypeScript** para el tipado, **Tailwind CSS** para el estilado, y los clientes oficiales `@supabase/ssr` y `@supabase/supabase-js` para hablar con la base. Se ejecuta como aplicación independiente del SGBD y consume las funciones almacenadas a través de PostgREST.

Su rol dentro del trabajo es doble. Por un lado, cubre el segundo párrafo de R1: ofrecer una interfaz desde la cual el personal del sistema puede consultar el log de auditoría. Por otro, sirve como vehículo de prueba para que el lector pueda observar la lógica almacenada en funcionamiento real (reservas, alquileres, devoluciones, mantenimientos) sin tener que invocar las funciones a mano desde `psql`. Todas las operaciones que realiza la aplicación pasan por los mismos `pa_*` y vistas documentados en este informe; la interfaz no contiene lógica de negocio propia, solo orquesta las llamadas y presenta los resultados.

Su diseño detallado, sus rutas, sus componentes y su experiencia de usuario quedan fuera del alcance de esta entrega, cuyo eje es la base de datos.

Arquitectura del schema

Los objetos DDL se agrupan en módulos numerados que se aplican en orden estricto. El orden refleja las dependencias topológicas: extensiones primero, tablas antes que constraints, funciones antes que triggers, permisos al final.

Para que el lector pueda identificar cada objeto por su nombre, el schema adopta una **convención de prefijos** uniforme: `fn_` para funciones, `pa_` para orquestadores (*procedimientos almacenados de alto nivel que coordinan varias operaciones —validaciones, escrituras y disparadores— bajo una sola llamada*), `trg_` para triggers y `vw_` para vistas. Esta convención se respeta en todo el catálogo que sigue.

Módulo	Contenido	Cantidad
<code>00_extensions.sql</code>	<code>pgcrypto</code> , <code>btree_gist</code> , <code>pg_cron</code>	3
<code>01_tables/</code>	Tablas de catálogos, negocio, auditoría	19
<code>02_constraints/</code>	Unique compuestos, FKs explícitas, <code>EXCLUDE USING gist</code>	14
<code>03_indexes/</code>	Índices <code>btree</code> , parciales y compuestos	11
<code>04_functions/</code>	Validaciones (<code>fn_*</code>), orquestadores (<code>pa_*</code>), ciclo de vida	22
<code>05_views/</code>	Vistas para consulta y reportes	7
<code>06_permissions/</code>	Roles, RLS, helpers, grants y <code>vw_usuario_legible</code>	11
<code>07_triggers/</code>	Triggers de auditoría + bloqueo de modificaciones	8
<code>08_seeds/</code>	Datos iniciales para los casos de uso	17

La separación entre `04_functions/` (lógica reutilizable y RPC) y `07_triggers/` (auditoría y ciclo de vida) responde a un criterio de mantenibilidad: las funciones de ciclo de vida del vehículo (`fn_alquiler_start`, `fn_alquiler_close`, `fn_mantenimiento_envio`, `fn_mantenimiento_devolucion`) viven en `04_functions/` junto con sus `CREATE TRIGGER` correspondientes, porque ambos elementos son inseparables del comportamiento de la entidad. En `07_triggers/` se ubican únicamente los triggers de auditoría genérica y el bloqueo que

mantiene a `audit_log` como bitácora de solo inserción, que son independientes de la lógica de negocio.

Catálogo de objetos DDL

Tablas (19)

A las 17 tablas modeladas en la Etapa 1 se suman dos tablas nuevas introducidas en Etapa 2: `audit_log` (R1) y `devolucion_vencida` (R9). La siguiente clasificación visual agrupa las tablas por rol funcional:

- **Catálogos:** `estado_vehiculo` , `tipo_vehiculo` , `tipo_reserva` .
- **Identidad:** `usuario` , `cliente` .
- **Negocio:** `sucursal` , `taller` , `vehiculo` , `tarifa` , `imagen_vehiculo` , `reserva` , `garantia_reserva` , `alquiler` , `factura` , `mantenimiento` .
- **Historiales y operativa:** `ubicacion_vehiculo` , `historial_estado_vehiculo` .
- **Auditoría y reportes (nuevas en Etapa 2):** `audit_log` , `devolucion_vencida` .

Tabla	Propósito	Archivo
<code>usuario</code>	Credenciales de acceso online (puente con <code>auth.users</code> de Supabase).	<code>01_tables/01_usuario.sql</code>
<code>cliente</code>	Datos personales del titular de alquileres y reservas.	<code>01_tables/02_cliente.sql</code>
<code>sucursal</code>	Sede física de la empresa donde residen los vehículos.	<code>01_tables/03_sucursal.sql</code>
<code>taller</code>	Proveedor externo de mantenimiento mecánico.	<code>01_tables/04_taller.sql</code>
<code>tipo_vehiculo</code>	Catálogo de carrocerías (Sedán, SUV, Cupé, etc.).	<code>01_tables/05_tipo_vehiculo.sql</code>
<code>estado_vehiculo</code>	Catálogo de estados operativos (<code>disponible</code> , <code>alquilado</code> , <code>en_mantenimiento</code> , ...).	<code>01_tables/06_estado_vehiculo.sql</code>
<code>tipo_reserva</code>	Catálogo de modalidades de reserva con los indicadores <code>requiere_garantia</code> y <code>antelacion_max_dias</code> .	<code>01_tables/07_tipo_reserva.sql</code>

Tabla	Propósito	Archivo
vehiculo	Unidad física que se alquila. Referencia origen, tipo y estado actual.	01_tables/08_vehiculo.sql
imagen_vehiculo	Entre 1 y 5 imágenes por vehículo (mínimo garantizado por aplicación, máximo por trigger).	01_tables/09_imagen_vehiculo.sql
ubicacion_vehiculo	Historial de sucursal donde se encuentra físicamente el vehículo.	01_tables/10_ubicacion_vehiculo.sql
historial_estado_vehiculo	Línea de tiempo de transiciones de estado.	01_tables/11_historial_estado_vehiculo.sql
tarifa	Precio diario y porcentaje de recargo por par (sucursal, tipo).	01_tables/12_tarifa.sql
reserva	Reserva online con validación de superposición.	01_tables/13_reserva.sql
garantia_reserva	Datos sensibles de la tarjeta de crédito que respalda una reserva.	01_tables/14_garantia_reserva.sql
alquiler	Alquiler efectivo. Lleva la máquina de estados principal del vehículo.	01_tables/15_alquiler.sql
mantenimiento	Envío y devolución a taller.	01_tables/16_mantenimiento.sql
factura	Comprobante con snapshot de tarifa al cierre del alquiler.	01_tables/17_factura.sql
audit_log (nueva)	Bitácora de solo inserción (no admite modificación ni borrado) con triple identidad (usuario_db , rol_sesion , usuario_app).	01_tables/18_audit_log.sql
devolucion_vencida (nueva)	Tabla histórica poblada por el job de detección de devoluciones tardías (R9).	01_tables/19_devolucion_vencida.sql

Constraints destacados

Más allá de las claves primarias y foráneas habituales, el schema utiliza restricciones avanzadas para hacer cumplir reglas de negocio a nivel de motor:

- **Antisuperposición temporal con `EXCLUDE USING gist`** sobre `alquiler` y `reserva` (`02_constraints/14_exclude_alquiler_reserva.sql`). El problema a resolver es el *doble alquiler*: dos solicitudes para el mismo vehículo en fechas que se pisan. La primera defensa es el trigger `fn_check_vehiculo_overlap` , que antes de insertar consulta si ya existe un período solapado; pero esa validación tiene dos pasos —consultar y después grabar— que no son atómicos, y por eso sufre una **condición de carrera** (*situación en la que el resultado depende del orden no determinístico en que se ejecutan operaciones simultáneas*): entre el `SELECT` que comprueba y el `INSERT` que graba hay una ventana en la que dos transacciones concurrentes no ven la fila que la otra todavía no confirmó, ambas concluyen que el vehículo está libre y ambas graban. Para cerrar esa ventana, la regla se traslada a una restricción que el motor valida **atómicamente al momento de escribir**. Una restricción `EXCLUDE` es una generalización de `UNIQUE` : en lugar de prohibir dos filas con la misma clave, prohíbe dos filas donde `id_vehiculo` sea **igual** (`WITH =`) y además sus rangos de fechas **se solapen** (`WITH &&`). El intervalo se construye con `tsrange(fecha_inicio, fecha_fin_prevista, '[' )` , semiabierto (incluye el inicio, excluye el fin), de modo que una devolución a las 10:00 y un nuevo alquiler a las 10:00 encadenen sin contar como superposición. La restricción se apoya en un índice **GiST** (*tipo de índice capaz de resolver operadores no triviales como el solapamiento de rangos, que un B-tree común no soporta*) y en la extensión `btree_gist` (*permite que una columna escalar como `id_vehiculo` conviva en el mismo índice GiST junto al rango temporal*). Las cláusulas `WHERE` la hacen parcial: solo bloquean los alquileres en estado `activo` y las reservas `pendiente` / `concretada` , de modo que un período cerrado o cancelado libera el vehículo. El resultado es que persistir dos períodos solapados del mismo vehículo se vuelve físicamente imposible, sin depender de que la capa de aplicación valide correctamente.
- **CHECK sobre `alquiler`** : `chk_alquiler_km` exige `km_inicio ≥ 0` y, cuando `km_fin` está informado, `km_fin ≥ km_inicio` (admite igualdad para alquileres sin kilometraje recorrido).
- **UNIQUE (`id_vehiculo`, `orden`)** sobre `imagen_vehiculo` : garantiza que no haya dos imágenes con la misma posición para el mismo vehículo.
- **UNIQUE (`id_sucursal`, `id_tipo`)** sobre `tarifa` : refuerza la cardinalidad declarada en Etapa 1.
- **Índice parcial único** sobre `ubicacion_vehiculo (id_vehiculo) WHERE fecha_hasta IS NULL` e idéntico patrón sobre `historial_estado_vehiculo` : garantizan un único registro vigente por vehículo.

Funciones de validación (`fn_*`)

En las tablas siguientes la columna *Tipo* distingue entre `FUNCTION` (lógica reutilizable, invocable desde otras funciones o desde HTTP) y *trigger* (*función que el motor ejecuta automáticamente antes o después de un `INSERT` / `UPDATE` / `DELETE` sobre una tabla, sin que el cliente la invoque explícitamente*).

Nombre	Propósito	Tipo
<code>fn_check_max_imagenes</code>	Limita a 5 imágenes por vehículo.	TRIGGER
<code>fn_check_vehiculo_overlap</code>	Validación complementaria de superposición para reservas y alquileres.	TRIGGER
<code>fn_alquiler_set_cerrado</code>	Setea <code>estado='cerrado'</code> cuando <code>fecha_devolucion_real</code> pasa de NULL a NOT NULL.	TRIGGER (BEFORE UPDATE)
<code>fn_alquiler_start</code>	Lifecycle al iniciar alquiler: estado <code>alquilado</code> , marca reserva como <code>concretada</code> .	TRIGGER (AFTER INSERT)
<code>fn_alquiler_close</code>	Lifecycle al cerrar: estado <code>disponible</code> , actualiza km, cierra y abre ubicación.	TRIGGER (AFTER UPDATE)
<code>fn_mantenimiento_envio</code>	Lifecycle al enviar vehículo a taller.	TRIGGER (AFTER INSERT)
<code>fn_mantenimiento_devolucion</code>	Lifecycle al recibir vehículo del taller.	TRIGGER (AFTER UPDATE)
<code>fn_calcular_factura</code>	Calcula días, recargo por horas excedidas y total, e inserta la fila en <code>factura</code> .	FUNCTION RETURNS BIGINT
<code>fn_handle_new_auth_user</code>	Puente Supabase Auth → <code>cliente</code> cuando se registra un nuevo usuario.	TRIGGER sobre <code>auth.users</code>
<code>fn_audit_generic</code>	Trigger genérico de auditoría que serializa la fila a <code>audit_log</code> .	TRIGGER
<code>fn_audit_log_append_only</code>	Rechaza UPDATE/DELETE sobre <code>audit_log</code> con <code>ERRCODE = 'insufficient_privilege'</code> .	TRIGGER
<code>fn_validar_periodo</code>	Valida coherencia de pares de fecha (inicio < fin, no en pasado); lanza excepción si no se cumple.	FUNCTION RETURNS VOID
<code>fn_validar_cliente_activo</code>	Verifica que el cliente exista; lanza excepción en caso contrario.	FUNCTION RETURNS VOID

Nombre	Propósito	Tipo
<code>fn_validar_vehiculo_operativo</code>	Verifica que el vehículo exista y esté <code>disponible</code> ; lanza excepción en caso contrario.	FUNCTION RETURNS VOID
<code>fn_auth_uid</code>	Lee el <code>JWT.sub</code> (UUID del usuario autenticado) desde <code>request.jwt.claims</code> ; devuelve <code>NULL</code> sin sesión.	FUNCTION RETURNS UUID
<code>fn_cliente_del_usuario</code>	Resuelve el <code>id_cliente</code> del usuario autenticado (<code>auth_user_id</code> → <code>cliente</code>).	FUNCTION RETURNS BIGINT
<code>fn_es_staff</code>	<code>TRUE</code> si el JWT trae <code>app_metadata.role = 'staff'</code> ; <code>FALSE</code> en cualquier otro caso.	FUNCTION RETURNS BOOLEAN

Los tres últimos helpers (`fn_auth_uid` , `fn_cliente_del_usuario` , `fn_es_staff`) residen en `06_permissions/02_rls_helpers.sql` por su acoplamiento con RLS y con el JWT de Supabase; el resto vive en `04_functions/` .

Procedimientos / orquestadores (`pa_*`)

Nombre	CU que implementa	Parámetros relevantes
<code>pa_registrar_cliente_con_usuario</code>	CU-01	IN datos cliente + credenciales; OUT <code>p_estado</code> , <code>p_mensaje</code> , <code>p_id_cliente</code> .
<code>pa_registrar_cliente_walkin</code>	Variante CU-01	Alta presencial sin sesión Auth; OUT <code>p_estado</code> , <code>p_mensaje</code> , <code>p_id_cliente</code> .
<code>pa_actualizar_cliente</code>	CRUD R3 (cliente)	El cliente autenticado modifica sus datos; el <code>id_cliente</code> se resuelve del JWT vía <code>fn_cliente_del_usuario()</code> . OUT <code>p_estado</code> , <code>p_mensaje</code> .
<code>pa_registrar_reserva</code>	CU-03	IN <code>p_id_cliente</code> , <code>p_id_vehiculo</code> , <code>p_id_tipo_reserva</code> , fechas; OUT estándar + <code>p_id_reserva</code> .

Nombre	CU que implementa	Parámetros relevantes
<code>pa_cancelar_reserva</code>	CU-04	IN <code>p_id_reserva</code> ; OUT estándar. Valida <code>estado='pendiente'</code> .
<code>pa_registrar_alquiler</code>	CU-05	IN <code>p_id_reserva</code> (<i>NULL para alquiler presencial</i>), cliente, vehículo, sucursal de retiro, <code>p_km_inicio</code> , fechas; OUT estándar + <code>p_id_alquiler</code> .
<code>pa_finalizar_alquiler</code>	CU-06	IN <code>p_id_alquiler</code> , <code>p_km_fin</code> , <code>p_id_sucursal_devolucion</code> y 3 IN opcionales para mantenimiento al cierre; OUT estándar + <code>p_id_factura</code> .
<code>pa_enviar_mantenimiento_programado</code>	CU-07	IN <code>p_id_vehiculo</code> , <code>p_id_taller</code> , <code>p_observaciones</code> ; OUT estándar (<code>p_estado</code> , <code>p_mensaje</code>). Valida que el vehículo esté en estado <code>disponible</code> antes de insertar la orden.
<code>pa_registrar_devolucion_mantenimiento</code>	CU-08	IN <code>p_id_vehiculo</code> , <code>p_km_salida_taller</code> (<i>NULL si no se reportan km</i>); OUT estándar (<code>p_estado</code> , <code>p_mensaje</code>). Localiza la orden abierta del vehículo automáticamente y, si se proveen km, valida que sean mayores o iguales a los actuales.
<code>pa_crear_vehiculo</code> / <code>pa_actualizar_vehiculo</code> / <code>pa_baja_vehiculo</code>	CRUD R3	CRUD completo de la entidad <code>vehiculo</code> con sus tres operaciones declaradas como <code>FUNCTION</code> independientes en <code>04_functions/19_pa_crud_vehiculo.sql</code> .
<code>pa_detectar_devoluciones_vehiculas</code>	Job R9	Sin parámetros. Único objeto declarado como <code>PROCEDURE</code> (lo invoca <code>pg_cron</code> con <code>CALL</code>).

En la implementación, el parámetro de salida que devuelve la clave del registro creado se llama uniformemente `p_id_generado` (excepto `pa_finalizar_alquiler` , que usa `p_id_factura`); en este catálogo se lo nombra `p_id_reserva` , `p_id_alquiler` , etc. para favorecer la legibilidad.

Decisión arquitectónica — R11: **FUNCTION** en lugar de **PROCEDURE**. PostgREST solo expone como endpoint RPC los objetos con `pg_proc.prokind = 'f'` (funciones). Por esa razón se reconvirtieron a **FUNCTION RETURNS RECORD** los doce orquestadores que el enunciado denomina genéricamente "procedimientos almacenados", manteniendo idéntica semántica: cada uno declara parámetros `OUT (p_estado, p_mensaje, p_id_*)` y envuelve la lógica en un bloque `BEGIN ... EXCEPTION WHEN ... END`. El único **PROCEDURE** real es `pa_detectar_devoluciones_vencidas`, porque su invocador es `pg_cron` (que ejecuta `CALL`) y no necesita ser expuesto a PostgREST. Esta decisión se justifica con detalle en los comentarios que acompañan al código del archivo `04_functions/07_pa_finalizar_alquiler.sql` y de los restantes `pa_*`.

Triggers

Nombre	Tabla	Evento	Propósito
<code>trg_audit_cliente</code>	<code>cliente</code>	INSERT / UPDATE / DELETE	Persiste en <code>audit_log</code> .
<code>trg_audit_vehiculo</code>	<code>vehiculo</code>	INSERT / UPDATE / DELETE	Persiste en <code>audit_log</code> .
<code>trg_audit_reserva</code>	<code>reserva</code>	INSERT / UPDATE / DELETE	Persiste en <code>audit_log</code> .
<code>trg_audit_alquiler</code>	<code>alquiler</code>	INSERT / UPDATE / DELETE	Persiste en <code>audit_log</code> .
<code>trg_audit_factura</code>	<code>factura</code>	INSERT / UPDATE / DELETE	Persiste en <code>audit_log</code> .
<code>trg_audit_mantenimiento</code>	<code>mantenimiento</code>	INSERT / UPDATE / DELETE	Persiste en <code>audit_log</code> .
<code>trg_audit_devolucion_vencida</code>	<code>devolucion_vencida</code>	INSERT / UPDATE / DELETE	Persiste en <code>audit_log</code> .
<code>trg_audit_log_no_update</code>	<code>audit_log</code>	BEFORE UPDATE OR DELETE	Lanza excepción <code>insufficient_privilege</code> — deja la tabla en modo de sólo inserción.

A estos se suman los triggers de ciclo de vida del vehículo, declarados en `04_functions/` junto con la función que ejecutan, por estar acoplados a la entidad de negocio:

`trg_alquiler_set_cerrado` (BEFORE UPDATE), `trg_alquiler_start` (AFTER INSERT), `trg_alquiler_close` (AFTER UPDATE), `trg_mantenimiento_envio` (AFTER INSERT) y `trg_mantenimiento_devolucion` (AFTER UPDATE).

Vistas (8) — todas nuevas en Etapa 2

Vista	Propósito	Tablas base
<code>vw_vehiculos_disponibles</code>	Catálogo público de unidades retirables por sucursal y tipo.	<code>vehiculo</code> , <code>ubicacion_vehiculo</code> , <code>tarifa</code> , <code>imagen_vehiculo</code> , <code>estado_vehiculo</code> .
<code>vw_alquileres_activos</code>	Consulta operativa de alquileres en curso con datos del cliente, vehículo y sucursal de retiro.	<code>alquiler</code> , <code>cliente</code> , <code>vehiculo</code> , <code>sucursal</code> .
<code>vw_reservas_pendientes</code>	Cola de reservas a confirmar para el staff.	<code>reserva</code> , <code>cliente</code> , <code>vehiculo</code> , <code>tipo_reserva</code> .
<code>vw_facturacion_mensual</code>	Agregados por mes y sucursal para reportes.	<code>factura</code> , <code>alquiler</code> , <code>vehiculo</code> , <code>sucursal</code> .
<code>vw_devoluciones_vencidas</code>	Panel R9 con datos enriquecidos del cliente y del vehículo.	<code>devolucion_vencida</code> , <code>cliente</code> , <code>vehiculo</code> .
<code>vw_audit_log_legible</code>	Consulta legible del log de auditoría con <code>tipo_op</code> traducido (<code>I</code> → <code>INSERT</code> , etc.) y fechas en zona horaria local.	<code>audit_log</code> .
<code>vw_stock_por_modelo</code>	Conteo público de unidades disponibles agrupadas por marca, modelo y año.	<code>vw_vehiculos_disponibles</code> (vista derivada).
<code>vw_usuario_legible</code>	Resuelve el UUID del actor de auditoría a un nombre legible para el panel de logs (gateada por <code>fn_es_staff()</code>).	<code>auth.users</code> , <code>cliente</code> .

Siete de estas vistas residen en `05_views/` ; `vw_usuario_legible` se ubica en `06_permissions/10_vw_usuario_legible.sql` porque depende de `fn_es_staff()` y del esquema `auth` de Supabase (solo se crea si `auth.users` existe).

Las vistas reducen consultas **N+1** (*patrón anti-rendimiento donde una consulta principal dispara N consultas adicionales, una por cada fila del resultado*), centralizan los `JOIN` s comunes y ofrecen una interfaz estable hacia los clientes HTTP: si una tabla base evoluciona (renombrar una columna, agregar un `id_*`), la vista absorbe el cambio sin que el código que la consume deba actualizarse.

Cron jobs (`pg_cron`)

pg_cron es la extensión que ejecuta sentencias SQL en horarios programados, equivalente a **cron** pero corriendo dentro del propio motor de PostgreSQL. El único job programado del sistema es la detección de devoluciones vencidas (R9):

- **Job:** `detectar-devoluciones-vencidas`
- **Frecuencia:** `0 */6 * * *` (cada 6 horas — 00:00, 06:00, 12:00, 18:00).
- **Comando:** `CALL pa_detectar_devoluciones_vencidas();`
- **Comportamiento:** inserta o actualiza en `devolucion_vencida` los alquileres con `estado='activo'`, `fecha_fin_prevista < NOW()` y `fecha_devolucion_real IS NULL`. La cláusula `ON CONFLICT (id_alquiler) DO UPDATE` garantiza la **idempotencia** (propiedad por la cual repetir la operación produce siempre el mismo resultado, aunque se ejecute varias veces) entre corridas sucesivas.
- **Registro defensivo:** el script `04_functions/21_schedule_jobs.sql` envuelve `cron.schedule()` en un bloque `DO ... EXCEPTION` que permite aplicar el schema en clusters donde **pg_cron** no esté disponible (sin abortar la aplicación completa del schema).

Seeds (17 archivos)

Los seeds en `08_seeds/` proveen un dataset funcional para los casos de uso: cinco usuarios online (`jperez` , `mgomez` , `lrodriguez` , `asanchez` , `cmartinez`) ligados a clientes, más dos clientes presenciales sin cuenta (Sofía López y Pedro Fernández); las cinco sucursales del escenario en el NEA (Posadas como sede central, Oberá, Puerto Iguazú, Corrientes y Resistencia); catálogos completos (5 tipos de vehículo, 5 estados, 3 tipos de reserva); diez vehículos con cinco imágenes cada uno (cincuenta en total, servidas desde el repositorio del proyecto en GitHub); una grilla de tarifas que cubre cada combinación sucursal × tipo presente; y seis reservas, cinco alquileres y tres mantenimientos que ejercitan los triggers de ciclo de vida y dejan datos consistentes para los reportes y la auditoría.

Implementación de los casos de uso

Esta sección describe cómo cada caso de uso de la Etapa 1 se materializa en objetos concretos del SGBD.

CU-03: Reservar vehículo online

- **Punto de entrada:** `pa_registrar_reserva(p_id_cliente, p_id_vehiculo, p_id_tipo_reserva, p_fecha_inicio, p_fecha_fin_prevista, OUT p_estado, OUT p_mensaje, OUT p_id_reserva)`.
- **Validaciones invocadas:** `fn_validar_periodo` , `fn_validar_cliente_activo` , `fn_validar_vehiculo_operativo` , `fn_check_vehiculo_overlap` .
- **Protección a nivel de motor:** la constraint `excl_reserva_overlap` cierra cualquier condición de carrera remanente.

- **Política de garantía:** si `tipo_reserva.requiere_garantia = true`, el flujo de aplicación exige el alta del registro paralelo en `garantia_reserva` antes de cerrar la transacción.
- **Retorno:** `p_estado ∈ {OK, ERROR_VALIDACION, ERROR_REFERENCIAL, ERROR_DUPLICADO}` + `p_mensaje` legible en castellano + `p_id_reserva` cuando el alta es exitosa.
- **Cumple:** R2 (excepciones + OUT), R5 (parámetros IN/OUT), R7 (validación de superposición).

CU-05: Iniciar alquiler (con o sin reserva)

- **Punto de entrada:** `pa_registrar_alquiler(p_id_reserva, p_id_cliente, p_id_vehiculo, p_id_sucursal_retiro, p_km_inicio, p_fecha_inicio, p_fecha_fin_prevista, OUT ...)`.
- **Alquiler presencial** (*walk-in*, en la jerga del rubro): si `p_id_reserva IS NULL`, el flujo presencial (R6) procede normalmente. Si está informado, el trigger `fn_alquiler_start` marca esa reserva como `concretada` al insertar.
- **Triggers disparados:** `trg_alquiler_start` (AFTER INSERT) lleva la máquina de estados del vehículo a `alquilado` y propaga el cambio a `vehiculo.id_estado`. Los triggers de auditoría dejan constancia en `audit_log`.
- **Cumple:** R5, R6, R7.

CU-06: Finalizar alquiler y generar factura

- **Orquestador:** `pa_finalizar_alquiler` realiza el `UPDATE` sobre `alquiler` con `fecha_devolucion_real`, `km_fin` e `id_sucursal_devolucion`.
- **Cadena de triggers:**
 - `trg_alquiler_set_cerrado` (BEFORE UPDATE) cambia `estado` a `cerrado` en la misma fila.
 - `trg_alquiler_close` (AFTER UPDATE) propaga `vehiculo.km_actuales = km_fin`, cierra el registro vigente en `historial_estado_vehiculo` y abre uno nuevo con estado `disponible`, cierra `ubicacion_vehiculo` y abre una entrada en la sucursal de devolución.
- **Facturación:** `fn_calcular_factura` recibe el `id_alquiler`, calcula días pactados, horas excedidas, recargo y total, copia los valores históricos de `precio_por_dia_aplicado` y `porcentaje_recargo_aplicado` desde la `tarifa` vigente, y emite la fila en `factura` con `numero_factura` correlativo.
- **Cumple:** R2, R5, R10 (registro de devolución + estado + recargo + factura, parte por trigger y parte por orquestador).

CU-07 / CU-08: Mantenimiento

- `pa_enviar_mantenimiento_programado` inserta en `mantenimiento`.
- `trg_mantenimiento_envio` (AFTER INSERT) lleva el vehículo a `en_mantenimiento`.
- `pa_registrar_devolucion_mantenimiento` actualiza `fecha_devolucion` y, opcionalmente, `vehiculo.km_actuales` con los kilómetros leídos a la salida del taller.

`trg_mantenimiento_devolucion` (AFTER UPDATE) devuelve el vehículo a `disponible`.

- **Cumple:** R5.

Auditoría — triple identidad (R1)

El requisito R1 exige registrar "qué usuario realizó cada operación". El punto de partida del diseño es una observación: en un entorno BaaS con Supabase + PostgREST no hay una sola respuesta a "quién hizo esto", sino tres, y cada una se puede falsificar con distinta facilidad. La más débil es el rol que la aplicación dice estar usando; la intermedia es la identidad humana que viaja firmada dentro del token JWT; y la más confiable es el rol con el que la conexión física se abrió contra el motor, porque ese no se puede alterar sin volver a autenticarse. Como ninguna basta por sí sola para una investigación forense, la tabla `audit_log` las guarda **las tres a la vez**.

A nivel de implementación, cada faceta se obtiene de una fuente distinta —algunas a partir de un **GUC** (*Grand Unified Configuration: parámetro de configuración de la sesión de PostgreSQL*, donde Supabase deposita los datos del JWT como `request.jwt.claims`)— y se almacena en una columna propia:

Campo	Quién lo escribe	Propiedad clave
<code>usuario_db</code>	<code>current_setting('request.jwt.claim.role')</code> o, en su defecto, <code>session_user</code> .	Rol Postgres efectivo (<code>authenticated</code> , <code>anon</code> , <code>service_role</code>). Lleva la semántica de privilegios aplicados pero es manipulable vía <code>SET ROLE</code> o GUC.
<code>rol_sesion</code>	<code>session_user</code> crudo.	Rol con el que se abrió físicamente la conexión al motor. No falsificable sin re-autenticarse: <code>SET ROLE</code> y los GUCs del JWT no lo afectan.
<code>usuario_app</code>	<code>auth.uid()</code> desde el JWT (UUID del usuario final en <code>auth.users</code>).	Identidad humana detrás de la operación. Resistente a manipulación gracias a la firma del JWT con la clave del proyecto; queda en <code>NULL</code> para invocaciones sin sesión HTTP (psql directo, <code>pg_cron</code>).

Solo inserción en dos niveles (el patrón se conoce como *append-only*). La tabla está protegida por:

1. Una política RLS `USING(FALSE)` para roles `authenticated` y `anon` (`06_permissions/04_rls_policies.sql`), que bloquea UPDATE/DELETE desde la API

PostgreSQL.

2. El trigger `trg_audit_log_no_update` (BEFORE UPDATE OR DELETE) ejecuta `RAISE EXCEPTION ... USING ERRCODE = 'insufficient_privilege'` para los roles con `BYPASSRLS` (e.g. `service_role`, `postgres` directo). Solo un superuser con `SET session_replication_role = replica` podría saltarlo, y ese comando es auditable a nivel de `pg_stat_statements`.

Operaciones registradas. `fn_audit_generic` distingue:

- `INSERT` → `valores_nuevos` con el `NEW` completo (`to_jsonb`) en formato JSONB.
- `UPDATE` → tanto `valores_anteriores` como `valores_nuevos`.
- `DELETE` → solo `valores_anteriores`.

Interfaz de consulta. Una interfaz auxiliar permite al personal autorizado revisar los registros de `audit_log` cruzados con `vw_audit_log_legible`, que traduce `tipo_op` (`I / U / D` → `INSERT / UPDATE / DELETE`) y formatea las fechas en zona horaria local. Esta consulta cumple el segundo párrafo de R1; el diseño concreto de su interfaz visual queda fuera del alcance de esta entrega.

Manejo de excepciones y transacciones (R2)

El proyecto adopta un **contrato uniforme** para todos los orquestadores `pa_*`:

- Todo `pa_*` declara tres parámetros de salida estándar: `OUT p_estado TEXT`, `OUT p_mensaje TEXT` y opcionalmente `OUT p_id_*` para devolver la clave del registro creado.
- `p_estado` se restringe al conjunto cerrado `{OK, ERROR_VALIDACION, ERROR_REFERENCIAL, ERROR_DUPLICADO, ERROR_ESTADO, ERROR}`.
- El cuerpo de cada función va envuelto en `BEGIN ... EXCEPTION WHEN unique_violation THEN ... WHEN foreign_key_violation THEN ... WHEN check_violation THEN ... WHEN OTHERS THEN ... END`, traduciendo cada `SQLSTATE` al código uniforme de `p_estado`.
- El cliente HTTP recibe siempre código `200 OK` con un cuerpo JSON como `{ "p_estado": "ERROR_VALIDACION", "p_mensaje": "El vehículo ya tiene una reserva en ese período" }`, en lugar de los habituales `500 Internal Server Error` con la traza de pila de Postgres expuesta.

Transacciones. PostgreSQL envuelve cada llamada RPC en una transacción implícita, de modo que el ciclo completo se narra en tres pasos. Cuando llega una llamada, el motor (1) abre la transacción y ejecuta la función. Al terminar, hay dos desenlaces posibles: (2) si la función llega al final sin lanzar un `RAISE`, el motor confirma los cambios (*commit*) automáticamente; (3) si en cambio se captura una `EXCEPTION` dentro del bloque, los cambios realizados antes del *raise* se revierten hasta el *savepoint* (*punto de guardado intermedio dentro de una transacción, que permite revertir solo a partir de ese punto sin abortar toda la operación*) implícito que el `BEGIN` creó al

entrar. Así el sistema cumple el espíritu de R2 —manejo explícito de COMMIT y ROLLBACK— sin que el código almacenado tenga que gestionar la transacción a mano.

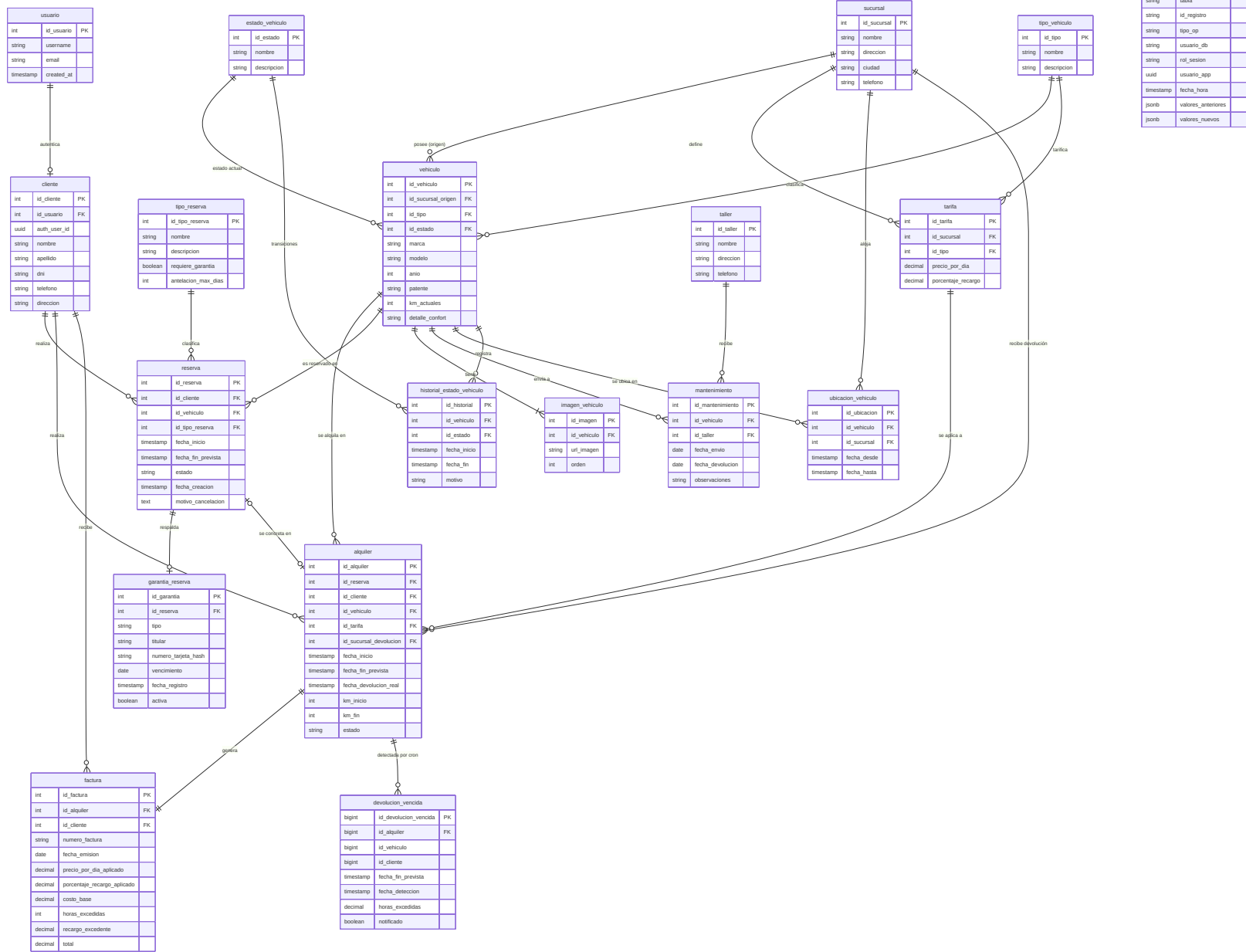
Mapeo de requerimientos del enunciado

La siguiente tabla cruza cada requerimiento numerado del enunciado (ver subsección anterior) con los objetos del schema que lo cumplen:

R	Requerimiento	Objetos que lo cumplen	Archivo
R1	Auditoría con triggers	<code>audit_log</code> + <code>fn_audit_generic</code> + 7 triggers <code>trg_audit_*</code> + <code>vw_audit_log_legible</code>	<code>01_tables/18_audit_log.sql</code> , <code>04_functions/12_fn_audit_generic.sql</code> , <code>07_triggers/01-07_*.sql</code>
R2	Excepciones + COMMIT/ROLLBACK	Patrón OUT (<code>p_estado</code> , <code>p_mensaje</code> , <code>p_id_*</code>) en todos los <code>pa_*</code>	<code>04_functions/</code> (todos los <code>pa_*</code>)
R3	CRUD entidades	<code>pa_crear_vehiculo</code> / <code>pa_actualizar_vehiculo</code> / <code>pa_baja_vehiculo</code> , <code>pa_registrar_cliente_con_usuario</code> , <code>pa_registrar_cliente_walkin</code>	<code>04_functions/19_pa_crud_vehiculo.sql</code> , <code>04_functions/11_*.sql</code> , <code>04_functions/22_*.sql</code>
R4	PL/pgSQL para insert/update/delete con retorno	Mismo patrón OUT documentado en R2	idem
R5	Parámetros IN/OUT	<code>pa_finalizar_alquiler</code> (6 IN + 3 OUT), <code>pa_registrar_reserva</code> , <code>pa_registrar_alquiler</code>	<code>04_functions/07_pa_finalizar_alquiler.sql</code> , <code>04_functions/16_pa_registrar_reserva.sql</code>
R6	Alquileres con/sin reserva	<code>pa_registrar_alquiler</code> acepta <code>p_id_reserva</code> opcional	<code>04_functions/18_pa_registrar_alquiler.sql</code>
R7	Procedimiento de reserva con validaciones	<code>pa_registrar_reserva</code> + <code>fn_check_vehiculo_overlap</code> + <code>excl_reserva_overlap</code>	<code>04_functions/16_pa_registrar_reserva.sql</code> , <code>02_constraints/14_*.sql</code>

R	Requerimiento	Objetos que lo cumplen	Archivo
R8	Cancelación de reservas con validaciones	<code>pa_cancelar_reserva (valida estado='pendiente' , marca garantia_reserva. activa= FALSE)</code>	<code>04_functions/17_pa_cancelar_reserva.sql</code>
R9	Jobs programados	<code>pg_cron</code> invoca <code>pa_detectar_devoluciones_vencidas</code> cada 6 h → <code>devolucion_vencida</code>	<code>04_functions/20_pa_detectar_devoluciones_vencidas.sql</code> , <code>04_functions/21_schedule_jobs.sql</code>
R10	Finalización de alquiler con triggers + programación	<code>pa_finalizar_alquiler</code> + <code>trg_alquiler_set_cerrado</code> + <code>trg_alquiler_close</code> + <code>fn_calcular_factura</code>	<code>04_functions/07_pa_finalizar_alquiler.sql</code> , <code>04_functions/03_fn_alquiler_lifecycle.sql</code> , <code>04_functions/05_fn_calcular_factura.sql</code>
R11	(Decisión del equipo) P PROCEDURE → FUNCTION por PostgREST	12 orquestadores migrados a FUNCTION RECORD	Comentarios incluidos en los archivos <code>04_functions/*.sql</code>

Diagrama ER actualizado (Etapa 2)



Respecto del DER de la Etapa 1, el modelo de Etapa 2 incorpora dos tablas nuevas — `audit_log` y `devolucion_vencida` — y formaliza la triple identidad de auditoría en las tres columnas `usuario_db`, `rol_sesion` y `usuario_app` ya descriptas. Ninguna entidad de negocio fue removida ni renombrada. El único ajuste a nivel de atributos derivó de delegar las credenciales en Supabase Auth: `usuario` ya no almacena `password_hash` (la contraseña vive en `auth.users`) y `cliente` incorpora `auth_user_id` —un UUID que actúa de puente con `auth.users`—; adicionalmente, `reserva` suma `motivo_cancelacion` para la trazabilidad de bajas (R8). Fuera de eso, el modelo conceptual sobrevivió a la implementación sin cambios estructurales, lo cual valida las decisiones tomadas en la Etapa 1.

Conclusiones de la Etapa 2

La implementación concreta del modelo confirmó la solidez del diseño conceptual: el modelo de la Etapa 1 sobrevivió sin cambios estructurales, y los únicos agregados fueron las dos tablas transversales (`audit_log` para R1 y `devolucion_vencida` para R9) y los ajustes de atributos que impuso Supabase Auth (`auth_user_id` en `cliente`, baja de `password_hash` en `usuario`). Esto es un indicador positivo de la calidad del modelado inicial: las decisiones de normalización, los catálogos y la separación entre pertenencia y ubicación física se mantuvieron exactamente como fueron pensadas.

Las decisiones arquitectónicas tomadas en esta etapa responden a problemas reales del entorno que solo aparecen al implementar: elegir Supabase como BaaS sobre PostgreSQL nativo, declarar los orquestadores como `FUNCTION` en lugar de `PROCEDURE` para exponerlos por PostgREST, modelar la auditoría con triple identidad para tener trazabilidad forense real ante manipulación, y resolver la antisuperposición temporal a nivel de índice con `EXCLUDE USING gist`. Cada una de ellas está justificada en los comentarios que acompañan al código en los archivos `.sql` correspondientes, de modo que la trazabilidad entre la decisión y el código que la materializa sea inmediata para el lector.

Hacia la Etapa 3 quedan tareas dentro de la propia capa de SGBD: refinar los índices según las consultas más frecuentes que aparezcan en producción, ampliar la cobertura de pruebas automatizadas sobre los flujos de error de los procedimientos almacenados (los flujos exitosos ya cuentan con pruebas), y evaluar el particionamiento de `audit_log` cuando el volumen lo justifique. La interfaz visual del sistema, aunque ya cuenta con una prueba de concepto funcional para consumir las RPC y las vistas, queda fuera del alcance de esta entrega. La capa de SGBD, en su forma actual, ya satisface la totalidad de los requerimientos R1–R10 del enunciado, más la decisión documentada del equipo para R11.

Etapla 3: presentación de la solución final

Objetivo y alcance de la etapa

La consigna define la tercera etapa como la "presentación de la solución final, lógica de negocio terminada en el SGBD más desarrollado en framework a elección, de las interfaces necesarias para el funcionamiento del sistema" (TFI 2026, sección *Etapas*, ítem 3). Respecto de la Etapa 2 —cuya consigna pedía lógica de negocio explícitamente *parcial*—, esta etapa introduce dos diferencias concretas:

1. **La lógica de negocio en el SGBD pasa de parcial a terminada:** se consolidan y cierran los pendientes que la propia conclusión de la Etapa 2 dejó enunciados dentro de la capa de base de datos.
2. **Se incorpora la capa de interfaz:** una aplicación construida sobre un framework a elección que expone las operaciones del sistema al usuario final. La consigna acota su alcance —"no abarcará la totalidad del sistema"—, de modo que esta capa no contiene reglas de negocio propias: se limita a invocar la lógica que ya reside en el motor.

El eje del trabajo sigue siendo la base de datos. La interfaz se documenta aquí en tanto evidencia de que cada caso de uso modelado en la Etapa 1 e implementado en la Etapa 2 es hoy operable de extremo a extremo, no como una pieza con valor propio sujeta a evaluación.

Cierre de la lógica de negocio en el SGBD

La capa de SGBD alcanzó en esta etapa su forma definitiva. El catálogo de objetos quedó conformado por **veinte tablas** (diecisiete de negocio más las tres transversales `audit_log`, `devolucion_vencida` y `resumen_mensual_sucursal`), **veinticinco funciones y orquestadores** en `04_functions/`, **ocho vistas** en `05_views/`, **ocho triggers** en `07_triggers/` y el árbol de permisos de `06_permissions/`. Respecto del catálogo documentado en la Etapa 2, los agregados de cierre fueron acotados: dos vistas de lectura — `vw_stock_por_modelo` (`05_views/08_vw_stock_por_modelo.sql`), que agrega el inventario disponible por marca, modelo y año para alimentar el catálogo público, y `vw_usuario_legible` (`06_permissions/10_vw_usuario_legible.sql`), que expone de forma controlada los datos de cuenta que la interfaz de perfil necesita sin filtrar columnas sensibles— y **dos tareas programadas de procesamiento masivo** —junto con la tabla de reporting `resumen_mensual_sucursal` que una de ellas materializa— descritas en las subsecciones siguientes.

La totalidad de los requerimientos R1 a R11 enumerados en el mapeo de requerimientos permanece satisfecha por los mismos objetos allí citados; esta etapa no removió ni redefinió ninguno. El cierre de la capa no alteró el modelo de negocio: los únicos agregados son dos vistas,

dos tareas programadas y una tabla de reporting derivada (`resumen_mensual_sucursal`) que consolida datos ya existentes sin redefinir ninguna entidad del dominio. Esto vuelve a confirmar, como ya lo había hecho la transición de la Etapa 1 a la 2, la solidez del modelo conceptual original.

Procesamiento masivo programado: expiración de reservas (R12)

A pedido de la cátedra, la entrega final incorpora un **procedimiento de procesamiento masivo accionado por un cron job**, que ejecuta varias actividades dentro de la base de datos en una sola corrida. La funcionalidad elegida resuelve un problema operativo concreto del escenario: la **higiene de reservas no concretadas**.

El problema. Una reserva en estado `pendiente` bloquea el vehículo en su período a través de la constraint `excl_reserva_overlap`. Si el cliente nunca se presenta a retirar la unidad (un *no-show*), esa reserva fantasma sigue impidiendo que otro cliente reserve —o que el personal alquile— el vehículo en esas fechas. Sin una limpieza periódica, el inventario realmente disponible se degrada con el tiempo.

La solución. El procedimiento `pa_expirar_reservas_vencidas` (`04_functions/24_pa_expirar_reservas_vencidas.sql`) recorre y cancela **en lote** todas las reservas que cumplen, simultáneamente:

- `estado = 'pendiente'` (ni concretada ni ya cancelada);
- `fecha_inicio < NOW() - INTERVAL '24 horas'` (la ventana de retiro venció hace más de la gracia de 24 h, que tolera demoras del mismo día);
- no existe ningún `alquiler` que referencie esa reserva (defensa en profundidad: el cliente efectivamente nunca retiró).

Sobre ese conjunto, una única sentencia con dos **CTE** (*Common Table Expression: subconsulta nombrada con `WITH`*; aquí, dos que además modifican datos) modificadoras ejecuta las **varias actividades** que exige la consigna, todas dentro de la misma transacción que `pg_cron` abre:

1. **Cancela las reservas:** `UPDATE reserva` lleva el `estado` a `cancelada` y graba un `motivo_cancelacion` con marca de tiempo y origen `sistema:cron`.
2. **Desactiva las garantías asociadas:** `UPDATE garantia_reserva` pone `activa = FALSE` para esas reservas, preservando el historial de tarjetas (no se borran filas) en línea con el criterio fiscal del resto del modelo.
3. **Audita la baja:** cada fila modificada dispara el trigger `trg_audit_reserva` ya existente, de modo que `audit_log` registra automáticamente la cancelación masiva sin código adicional en el procedimiento.

Al liberarse las reservas, la constraint de superposición deja de bloquear esos períodos y el vehículo vuelve a ofrecerse en el catálogo, cerrando el ciclo.

Programación y aislamiento. La tarea se registra en `pg_cron` (`04_functions/21_schedule_jobs.sql`) con la expresión `0 3 * * *` —una corrida diaria a las 03:00, en horario de baja actividad— y convive con la tarea de detección de devoluciones vencidas (R9) bajo guardas de idempotencia independientes por `jobname`. El procedimiento se declara `SECURITY DEFINER` con `search_path` fijo y, replicando el criterio de seguridad de R9, se le **revoca el permiso de ejecución** a los roles `authenticated` y `anon` (`06_permissions/08_grants_jobs.sql`): es una tarea interna de mantenimiento, no una API pública, y no debe poder dispararse desde PostgREST. Su ejecución es idempotente: una reserva ya cancelada deja de cumplir el filtro de estado, por lo que corridas sucesivas no la reprocesan.

Esta funcionalidad se incorpora al mapeo de requerimientos como **R12**.

Cierre contable mensual de facturación (R13)

La segunda tarea masiva responde a una necesidad de negocio distinta —de reporting, no de higiene de datos— y muestra una variante complementaria del procesamiento por lotes: en lugar de modificar muchas filas, **agrega muchas filas en una sola** y materializa el resultado.

El problema. La vista `vw_facturacion_mensual` (Etapa 2) calcula el agregado de facturación por mes y sucursal recorriendo `factura`, `alquiler` y `vehiculo` en cada consulta. Es adecuada para el mes en curso, pero recalcular una y otra vez meses ya cerrados —cuyos números son inmutables— es costoso e innecesario para tableros gerenciales e informes históricos.

La solución. El procedimiento `pa_cerrar_facturacion_mensual` (`04_functions/25_pa_cerrar_facturacion_mensual.sql`) ejecuta, una vez por mes, un **cierre contable** del período recién terminado: una única sentencia `INSERT ... SELECT ... GROUP BY` recorre todas las facturas del mes, las une con su alquiler y vehículo, agrega por sucursal y **consolida una fila por sucursal** en la tabla nueva `resumen_mensual_sucursal` (`01_tables/20_resumen_mensual_sucursal.sql`). Cada fila congela: cantidad de facturas emitidas, total de costo base, total de recargos, total facturado y kilómetros recorridos en el mes.

Decisiones de diseño que mantienen la coherencia del modelo:

- **Atribución de sucursal idéntica a la vista:** se imputa a la sucursal de origen del vehículo (`vehiculo.id_sucursal_origen`), el mismo criterio que `vw_facturacion_mensual`, de modo que el cierre materializado y la vista en vivo jamás se contradicen.
- **Período cerrado, no actual:** la corrida del día 1 consolida el mes calendario anterior (`DATE_TRUNC('month', CURRENT_DATE - INTERVAL '1 month')`), ya completo.
- **Idempotencia por UPSERT:** la unicidad (`periodo, id_sucursal`) (`uq_resumen_periodo_sucursal`) permite que `ON CONFLICT ... DO UPDATE` refresque el cierre de un mes si se re-ejecuta, sin duplicar filas. La columna `fecha_cierre` registra cuándo se consolidó.

Programación y aislamiento. La tarea se registra en `pg_cron` con la expresión `0 4 1 * *` —el día 1 de cada mes a las 04:00, después de la expiración diaria— y, como las demás tareas internas, se declara `SECURITY DEFINER` y se le revoca la ejecución a `authenticated / anon` (`06_permissions/08_grants_jobs.sql`). La tabla `resumen_mensual_sucursal` tiene RLS habilitada con la política `resumen_mensual_sucursal_staff_read`: solo el personal la lee, y la escritura queda revocada para todos los roles de cliente, de modo que la única vía de alta es la propia tarea (que corre como `postgres`). Sobre esa lectura restringida se monta la interfaz gerencial descrita más abajo (`/admin/reportes-mensuales`).

Esta funcionalidad se incorpora al mapeo de requerimientos como **R13**. Con R12 y R13, la entrega final suma dos procesamiento masivos programados —uno que modifica datos en lote (expiración de reservas) y otro que los consolida en lote (cierre contable)— y completa el conjunto de lógica de negocio del SGBD.

La capa de interfaz: arquitectura y alcance

El framework elegido fue **Next.js** (React) con **App Router** (el enrutador de Next.js basado en el árbol de carpetas de `app/`, donde cada carpeta es un segmento de URL y cada `page.tsx` la vista que la renderiza), TypeScript y Tailwind CSS para el estilado. La aplicación reside en `frontend/` y se comunica con el motor exclusivamente a través de la API que Supabase expone: invoca los orquestadores `pa_*` como llamadas **RPC** sobre PostgREST y lee las vistas `vw_*` como recursos REST. No existe en la interfaz ninguna sentencia SQL ni regla de negocio: cada acción del usuario se traduce en una única llamada a una función almacenada, y cada listado en una consulta a una vista.

Esta disciplina se materializa en un único punto de contacto tipado con el motor. El helper `rpcCall` (`frontend/lib/supabase/rpc.ts`) centraliza toda invocación de orquestadores y verifica en tiempo de compilación que los argumentos de cada llamada coincidan con la firma declarada en `frontend/types/database.ts`, un reflejo del schema real de la base. De este modo, la frontera entre la aplicación y el SGBD queda controlada en un solo archivo y cualquier divergencia respecto de la firma de un `pa_*` se detecta antes de ejecutar.

La aplicación distingue dos audiencias, alineadas con los actores de la Etapa 1: un área **pública y de cliente registrado** (catálogo, reservas, perfil) y un área **de personal** (`/admin/*`) para la operación presencial. La separación no es solo de navegación: se sostiene en las políticas RLS y los roles del motor descritos más abajo, de modo que la pertenencia de un usuario a un área no depende de la interfaz sino de su identidad ante la base.

Módulos implementados

Los módulos desarrollados, definidos durante el cursado, cubren el ciclo de vida completo del negocio. La siguiente tabla relaciona cada módulo de la interfaz con su ruta en la aplicación, los objetos del SGBD que invoca y el caso de uso o requerimiento que satisface.

Módulo	Ruta(s)	Objeto(s) del SGBD	CU / R
Catálogo de vehículos	/ , /vehiculos/[id_vehiculo]	vw_vehiculos_disponibles , vw_stock_por_modelo	CU-01/02
Autenticación	/login	Supabase Auth (GoTrue + JWT)	—
Registro y perfil	/perfil	pa_registrar_cliente_con_usuario , pa_actualizar_cliente , vw_usuario_legible	R3
Reserva online	/reservar/[id_vehiculo]	pa_registrar_reserva	CU-03, R7
Mis reservas y cancelación	/mis-reservas	vw_reservas_pendientes , pa_cancelar_reserva	CU-04, R8
Gestión de flota (CRUD)	/admin/vehiculos	pa_crear_vehiculo , pa_actualizar_vehiculo , pa_baja_vehiculo	R3
Alta de alquiler	/admin/alquileres/nuevo , /admin/alquileres	pa_registrar_alquiler , pa_registrar_cliente_walkin , vw_alquileres_activos , vw_reservas_pendientes	CU-05, R5/R6
Cierre de alquiler y factura	/admin/alquileres/[id_alquiler]/cerrar	pa_finalizar_alquiler → fn_calcular_factura	CU-06, R10
Facturación	/admin/facturas , /admin/facturas/[id_factura]	vw_facturacion_mensual	R10
Envío a mantenimiento	/admin/mantenimientos/nuevo , /admin/mantenimientos	pa_enviar_mantenimiento_programado	CU-07
Devolución de mantenimiento	/admin/mantenimientos/[id_mantenimiento]/devolucion	pa_registrar_devolucion_mantenimiento	CU-08

Módulo	Ruta(s)	Objeto(s) del SGBD	CU / R
Devoluciones vencidas	/admin/devoluciones-vencidas	vw_devoluciones_vencidas	R9
Reporte contable mensual	/admin/reportes-mensuales	resumen_mensual_sucursal (← pa_cerrar_facturacion_mensual)	R13
Auditoría	/admin/auditoria , /admin/auditoria/[id]	vw_audit_log_legible	R1

En total, la interfaz consume **doce orquestadores** `pa_*` y **ocho vistas** `vw_*`. La correspondencia es exhaustiva en ambos sentidos: no hay pantalla que invoque una función inexistente, ni operación de negocio del motor que carezca de un punto de entrada en la interfaz. La única excepción deliberada es `pa_detectar_devoluciones_vencidas`, que no se invoca desde la aplicación porque su disparo es automático: `pg_cron` lo ejecuta cada seis horas (R9) y el resultado se consulta desde la pantalla de devoluciones vencidas a través de `vw_devoluciones_vencidas`. Es decir, la interfaz observa el efecto del job, no lo dispara.

Recorrido funcional de los casos de uso

Los cuatro casos de uso principales identificados en la Etapa 1 se ejecutan sobre el sistema vivo del siguiente modo:

- **CU-03 — Reservar vehículo online:** el cliente registrado parte del catálogo (`vw_vehiculos_disponibles`), elige una unidad y un período en `/reservar/[id_vehiculo]`, y la interfaz invoca `pa_registrar_reserva`. Toda la validación —período, cliente activo, vehículo operativo y, sobre todo, la antisuperposición temporal reforzada por la constraint `excl_reserva_overlap`— ocurre en el motor. La interfaz se limita a mostrar el `p_mensaje` legible que el orquestador devuelve.
- **CU-05 — Iniciar alquiler (retiro):** el personal opera `/admin/alquileres/nuevo`. Si el alquiler proviene de una reserva, la toma del listado `vw_reservas_pendientes`; si es presencial (*walk-in*), da de alta al cliente en el acto con `pa_registrar_cliente_walkin`. En ambos casos cierra con `pa_registrar_alquiler`, que registra `km_inicio` y dispara la transición del vehículo a `alquilado`.
- **CU-06 — Finalizar alquiler y generar factura:** desde `/admin/alquileres/[id_alquiler]/cerrar`, el personal informa `km_fin`, fecha de devolución real y sucursal de entrega; `pa_finalizar_alquiler` orquesta el cierre, y `fn_calcular_factura` calcula días, recargo por horas excedidas y total, emitiendo la factura con el *snapshot* de tarifa. La interfaz solo presenta el comprobante resultante.

- **CU-07/08 — Mantenimiento:** el envío al taller se registra en `/admin/mantenimientos/nuevo` (`pa_enviar_mantenimiento_programado` , que lleva el vehículo a `en_mantenimiento`) y la devolución en `/admin/mantenimientos/[id_mantenimiento]/devolucion` (`pa_registrar_devolucion_mantenimiento` , que lo devuelve a `disponible`).

En cada recorrido la propiedad invariante es la misma: la interfaz no decide nada. Recibe datos del usuario, los envía a un orquestador y muestra la respuesta. La autoridad sobre el estado del sistema permanece íntegramente en el SGBD.

Seguridad de la capa de interfaz

La separación entre el área pública/cliente y el área de personal no se confía a la navegación de la aplicación, sino que se apoya en los mecanismos del motor descritos en la Etapa 2. La autenticación la provee Supabase Auth, que emite un **JWT** firmado por sesión; ese token viaja en cada llamada y el motor lo usa para resolver la identidad (`auth.uid()`) y el rol efectivo. Sobre esa base, las políticas **RLS** definidas en `06_permissions/` garantizan que un cliente solo pueda ver y operar sus propios alquileres y reservas, con independencia de lo que la interfaz le ofrezca en pantalla. La consecuencia de diseño es que aun una interfaz mal construida —o un cliente HTTP que la sortee por completo— no puede acceder a datos ajenos: el filtrado ocurre en el motor, no en la aplicación. La pantalla de auditoría (`/admin/auditoria`) hereda esta misma lógica y queda reservada al personal autorizado, cumpliendo el segundo párrafo de R1.

Conclusiones de la Etapa 3

El trabajo cierra su ciclo completo: del modelo conceptual de la Etapa 1, a la implementación de la lógica en el motor en la Etapa 2, a un sistema operable de extremo a extremo en la Etapa 3. La interfaz construida confirma empíricamente lo que el modelo de datos y los procedimientos almacenados prometían: cada requisito del escenario —reservar con validación de superposición, alquilar registrando kilómetros, facturar con recargo por demora, gestionar el ciclo de mantenimiento y auditar cada operación— es hoy ejecutable por un usuario real.

La decisión arquitectónica de fondo —mantener toda la lógica de negocio dentro del SGBD y reducir la interfaz a un consumidor de RPC y vistas— se reveló acertada en la práctica: permitió delegar la construcción de la capa visual sin riesgo para la pieza evaluada, porque ninguna regla de negocio vive fuera del motor. El sistema entregado satisface la totalidad de los requerimientos R1–R11 del enunciado y los expone, módulo por módulo, a través de las interfaces necesarias para su funcionamiento, tal como exige la consigna de esta etapa.

Referencias

[1] R. Elmasri y S. B. Navathe, Fundamentos de Sistemas de Bases de Datos, 5.^a ed. Madrid, España: Pearson/Addison Wesley, 2007.